



BASE24[®]

ITS Kernel Files Maintenance Manual

© 2000 by ACI Worldwide Inc. All rights reserved.

All information contained in this document is confidential and proprietary to ACI Worldwide Inc. This material is a trade secret and its confidentiality is strictly maintained. Use of any copyright notice does not imply unrestricted or public access to these materials. No part of this document may be photocopied, electronically transferred, modified, or reproduced in any manner without the prior written consent of ACI Worldwide Inc.

ACI, BASE24, BASE24-telebanking, and NET24-XPNET are trademarks or registered trademarks of ACI Worldwide Inc., Transaction Systems Architects, Inc., or their subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Print Record

07/2000, Version 3 (Reissue)

03/97, Version 2 (Reissue)

07/95, Version 1 (New)

Contents

Preface	ix
Conventions Used in This Manual	xvii
Section 1	
Introduction	1-1
ITS Kernel Files and Functions	1-2
Standard ITS Kernel Files	1-2
Process Control Functions	1-3
File Access	1-4
Function Keys	1-5
Default Records	1-6
Customer-Configured Data	1-7
Data Entry Hierarchy	1-8
Basic Files	1-8
Relationship Files	1-9
File Relationships and Dependencies	1-9
Process Control Function Dependencies	1-10
Help Screens	1-12
Object and Extended Memory Table Abbreviations	1-14
Object Name Abbreviations	1-14
Extended Memory Table Name Abbreviations	1-16
Section 2	
Assign File (ASGN)	2-1
Screen 1	2-2
Default Records	2-3

Section 3

Assign/Object Relationship File (AORF)	3-1
Screen 1 Function Keys	3-2
Screen 1	3-3
Default Records	3-5

Section 4

Assign/Process Type Relationship File (APRF)	4-1
Screen 1 Function Keys	4-2
Screen 1	4-3
Default Records	4-5

Section 5

Assign Warmboot (AWB) Screen	5-1
Screen 1 Function Keys	5-3
Screen 1	5-4
Screen 2 Function Keys	5-6
Screen 2	5-7

Section 6

Configuration Control (CCS) Screen	6-1
Reallocating a Newly Built or Rebuilt Extended Memory Table	6-3
Object-Oriented Processes	6-3
Procedural Processes	6-3
Pathway Environment	6-4
XPNET Environment	6-4
Screen 1 Function Keys	6-6
Screen 1	6-7
Screen 2 Function Keys	6-10
Screen 2	6-11

Section 7

Debug (DBG) Screen	7-1
Screen 1	7-3

Section 8

EMT/Object Relationship File (EORF)	8-1
Screen 1 Function Keys	8-2
Screen 1	8-3
Default Records	8-5

Section 9

EMT/Process Type Relationship File (EPRF)	9-1
Screen 1 Function Keys	9-2
Screen 1	9-3
Default Records	9-5

Section 10

EMT Warmboot (EWB) Screen	10-1
Screen 1 Function Keys	10-3
Screen 1	10-4
Screen 2 Function Keys	10-6
Screen 2	10-7

Section 11

Event Suppression File (ESF)	11-1
Screen 1	11-3

Section 12

Extended Memory Table File (EMT)	12-1
Screen 1	12-3
Default Records	12-5

Section 13

LCONF Warmboot (LWB) Screen	13-1
Screen 1 Function Keys	13-3
Screen 1	13-4

Section 14

Log Permission Process Partition File (LPF)	14-1
Screen 1	14-3

Section 15

Object File (OBJ)	15-1
Screen 1	15-3
Screen 2	15-8
Default Records	15-11

Section 16

Param Warmboot (PWB) Screen	16-1
Screen 1 Function Keys	16-3
Screen 1	16-4

Section 17

Process File (PRO)	17-1
Screen 1	17-2

Section 18

Process/Object Relationship File (PORF)	18-1
Screen 1 Function Keys	18-2
Screen 1	18-3

Section 19

Process Type File (PTF)	19-1
Screen 1	19-3
Default Records	19-5

Section 20

Server Class Type/Process Relationship File (SPRF)	20-1
Screen 1 Function Keys	20-2
Screen 1	20-3

Section 21

Server Class Type File (SCT)	21-1
Screen 1	21-3

Section 22

Server Control (SCS) Screen	22-1
Screen 1	22-3
Screen 2 Function Keys	22-5
Screen 2	22-6

Section 23

Source Class Map File (SCM)	23-1
Screen 1	23-3

Section 24

Status (STAT) Screen	24-1
Screen 1	24-3
Screen 2 Function Keys	24-4
Screen 2	24-5
Status Command Response Data	24-7

Section 25

Trace Control (TRC) Screen	25-1
Trace Levels	25-3
Trace File Interpretation	25-4
Screen 1 Function Keys	25-5
Screen 1	25-6
Screen 2 Function Keys	25-7
Screen 2	25-8
Screen 3 Function Keys	25-11
Screen 3	25-12
Screen 4 Function Keys	25-15
Screen 4	25-16
Screen 5 Function Keys	25-18

Trace Control (TRC) Screen *continued*

Screen 5.....	25-19
Screen 6 Function Keys	25-21
Screen 6.....	25-22
Index	Index-1
Index by Field Name	Index-5
Index by Data Name	Index-7

Preface

This manual provides descriptions of the BASE24 integrated transaction services (ITS) kernel files maintenance and operations screens. The information in this manual enables users to enter and update records in the ITS kernel files as well as perform ITS kernel network operation functions correctly. BASE24 ITS kernel files and operator functions are accessed from the KERNEL entry on the Virtual Menu and consist of the files and operator functions used exclusively with the ITS kernel.

The screen descriptions in this manual include information about the purpose of the fields on the screen, the codes which can be placed in these fields, and their default values and lengths.

Audience

This manual is intended as a reference source for institution or service provider personnel responsible for updating and maintaining the BASE24 ITS kernel database as well as for operating the ITS kernel environment.

Prerequisites

Readers should be familiar with the ***BASE24 CRT Access Manual*** before reading this manual. The ***BASE24 CRT Access Manual*** includes information about logging on to BASE24, accessing the appropriate screens, and using function keys.

This manual should be read in conjunction with the ***BASE24 ITS Kernel Configuration Manual***. Many of the procedures presented in the ***BASE24 ITS Kernel Configuration Manual*** use the file maintenance screens presented in this manual.

Readers may also find the ***BASE24 ITS Kernel Objects Reference Manual*** and ***BASE24 IEP Objects Reference Manual*** helpful when configuring objects on some file maintenance screens.

Additional Documentation

The BASE24 documentation set is arranged so that each manual presents a topic or group of related topics in detail. When one manual presents a topic that has already been covered in detail in another manual, the topic is summarized and the reader is directed to the other manual for additional information. Information has been arranged in this manner to be more efficient for readers who do not need the additional detail and at the same time provide the source for readers who require the additional information. This manual contains references to the following BASE24 publications:

- The ***BASE24 CRT Access Manual*** contains information on accessing ITS kernel file maintenance and process control screens. This manual describes how screens are accessed in a BASE24 environment.
- The ***BASE24 Logical Network Configuration File Manual*** describes the Logical Network Configuration File (LCONF) assigns and params used by ITS processes that must be configured in certain ITS files.
- The ***BASE24 ITS Kernel Configuration Manual*** describes the steps involved in configuring an application to the ITS environment.
- The ***BASE24 ITS Kernel Objects Reference Manual*** describes the ITS kernel objects. These objects are used when building any object-oriented process running in the ITS environment.
- The ***BASE24 IEP Configuration Manual*** describes the steps involved in configuring an Integrated Endpoint process to the ITS environment.
- The ***BASE24 IEP Objects Reference Manual*** describes the Integrated Endpoint process objects. These objects are used when building an Integrated Endpoint process in the ITS environment.
- The ***BASE24 Text Command Reference Manual*** contains detailed information on the DELIVER PROCESS command, which is used to reinitialize processes that use extended memory tables in the XPNET environment.

In addition to the above BASE24 publications, the ***BASE24 ITS Kernel Files Maintenance Manual*** contains references to the ***NET24-XPNET Network Control Command Reference Manual***, which describes process and station commands and attributes.

In addition to the above BASE24 and XPNET publications, the ***BASE24 ITS Kernel Files Maintenance Manual*** contains references to the following Tandem publications:

- The ***NonStop TS/MP System Management Manual*** describes commands used to reinitialize Pathway servers that use extended memory tables in the ITS environment.
- The ***Binder Manual*** contains detailed information on the Binder product, which can be used to determine the objects that make up an executable process.
- The ***Debug Manual*** contains detailed information on the Debug product, a debugging tool that can be accessed using the Debug screen if your ITS processes support it.
- The ***INSPECT Manual*** contains detailed information on the INSPECT product, a debugging tool that can be accessed using the Debug screen if your ITS processes support it.

Software

This manual documents standard processing for Integrated Transaction Services release 1.3.

Customer-specific code modifications (that is, CSMs) may result in processing that differs from the material presented in this manual. The customer is responsible for identifying and noting these changes.

Manual Summary

The following is a summary of the contents of this manual.

“Conventions Used in This Manual” follows this preface and describes notation and documentation conventions necessary to understand the information in the manual.

Section 1, “Introduction,” introduces the integrated transaction services (ITS) configuration file maintenance configuration screens and process control screens. This section also includes information on accessing screens and screen functions and provides an overview of what data is provided by ACI on a default basis and what data must be configured by the customer during or after installation.

Section 2, “Assign File (ASGN),” describes the screen used to maintain the ASGN. The ASGN configures Logical Network Configuration File (LCONF) assigns that can be reread using the ASSIGN WARMBOOT command.

Section 3, “Assign/Object Relationship File (AORF),” describes the screen used to maintain the AORF. The AORF defines relationships between LCONF assigns that can be reread and ITS objects.

Section 4, “Assign/Process Type Relationship File (APRF),” describes the screen used to maintain the APRF. The APRF defines relationships between LCONF assigns that can be reread and ITS process types.

Section 5, “Assign Warmboot Screen (AWB),” describes the AWB process control screen used to send the ASSIGN WARMBOOT command to objects to reread specified LCONF assigns.

Section 6, “Configuration Control Screen (CCS),” describes the CCS process control screen used to send EMT Build commands to the Extended Memory Table Build utility.

Section 7, “Debug Screen (DBG),” describes the DBG process control screen used to run a particular ITS process or Queue process under control of the Tandem debugging tools, Debug or Inspect, for debugging purposes.

Section 8, “EMT/Object Relationship File (EORF),” describes the screen used to maintain the EORF. The EORF defines relationships between all objects and individual extended memory tables that can be reallocated using the EMT WARMBOOT command.

Section 9, “EMT/Process Type Relationship File (EPRF),” describes the screen used to maintain the EPRF. The EPRF defines relationships between all extended memory tables that can be reallocated using the EMT WARMBOOT command and all process types that can access these tables.

Section 10, “EMT Warmboot Screen (EWB),” describes the EWB process control screen used to send the EMT WARMBOOT command to processes to allocate specified extended memory tables.

Section 11, “Event Suppression File (ESF),” describes the screen used to maintain the ESF. The ESF defines events that are to be suppressed. The file contains one record for each event that is to be suppressed in some way.

Section 12, “Extended Memory Table (EMT),” describes the screen used to maintain the EMT. The EMT defines all extended memory tables that can be built using the Extended Memory Table Build utility or allocated using the EMT WARMBOOT command.

Section 13, “LCONF Warmboot Screen (LWB),” describes the LWB process control screen used to send the LCONF WARMBOOT command to selected objects. This command causes objects to be reinitialized. Reinitialization causes all LCONF assigns and params to be reread and all extended memory tables to be reallocated.

Section 14, “Log Permission Process Partition File (LPF),” describes the screen used to maintain the LPF. The LPF maps ranges of subsystem identifiers (SSIDs) to specific Log Permission processes responsible for arbitration when determining whether an event is to be logged or suppressed at the network level.

Section 15, “Object File (OBJ),” describes the screen used to maintain the OBJ. The OBJ defines all objects used in the ITS environment and their process control capabilities.

Section 16, “Param Warmboot Screen (PWB),” describes the PWB process control screen used to sent the PARAM WARMBOOT command to objects to reread specified LCONF params.

Section 17, “Process File (PRO),” describes the screen used to maintain the PRO. The PRO defines all processes used in the ITS environment, including both ACI- and customer-defined ITS processes as well as procedural processes that can receive process control commands.

Section 18, “Process/Object Relationship File (PORF),” describes the screen used to maintain the PORF. The PORF defines relationships between ITS processes and objects.

Section 19, “Process Type File (PTF),” describes the screen used to maintain the PTF. The PTF defines all process types used in the ITS environment. This includes types for all ACI- and customer-defined ITS processes as well as procedural processes that can receive process control commands.

Section 20, “Server Class Type/Process Relationship File (SPRF),” describes the screen used to maintain the SPRF. The SPRF defines relationships between server class types and processes in the ITS environment that can receive process control commands.

Section 21, “Server Class Type File (SCT),” describes the screen used to maintain the SCT. The SCT defines all server class types used in the ITS environment to the process control subsystem, including both ACI- and customer-defined server class types.

Section 22, “Server Control Screen (SCS),” describes the SCS process control screen used to send general server control commands to processes in the ITS environment.

Section 23, “Source Class Map File (SCM),” describes the screen used to maintain the SCM. The SCM relates process and station source class values in the XPNET system with message handler objects in the ITS environment.

Section 24, “Status Screen (STAT),” describes the STAT process control screen used to send the STATUS command to objects within an ITS process.

Section 25, “Trace Control Screen (TRC),” describes the TRC process control used to start, stop, and inquire about traces executed on objects in the ITS environment.

Readers can use the index by field name to locate information about a particular screen field and the index by data name to locate information about a particular field from a file or record structure.

Publication Identification

Two entries appearing at the bottom of each page uniquely identify this BASE24 publication. The publication number (for example, OK-AE000-02 for the ***BASE24 ITS Kernel Files Maintenance Manual***) appears on every page to assist readers in applying updates to the appropriate manual. The publication date (for example, 07/2000 for July, 2000) identifies the last time material on a particular page has been revised. This date can be used to verify that the reader has the most current information and is particularly helpful when dealing with HELP24 to answer operating questions.

The print record, located on the page immediately following the title page, is also helpful when verifying that a manual contains the most current information. An entry is added to the print record each time a BASE24 publication is modified in any way. Each print record entry includes the date of the change, a unique version number, and the type of change (new, update, reissue, or preliminary). Version 1 is assigned to a *new* manual. Later versions can be updates or reissues, depending on the changes being made. An *update* typically involves changes on a small number of pages throughout the manual. A *reissue* occurs when all pages of the manual

are replaced because changes affect a large number of pages. A manual is identified as *preliminary* when it is released to customers on a limited basis, but it has not yet reached final form.

ACI Worldwide Inc.

Conventions Used in This Manual

This section explains how field descriptions and unlabeled fields are documented in this manual. It also explains a change in Nucleus process terminology.

Field Descriptions

Each field appearing on a file maintenance or process control screen is listed by name and then described. Field descriptions summarize the contents, purpose, and permissible values, as shown in the following example taken from the Process Type File (PTF).

Example

PROCESS TYPE — An acronym defining a particular category of processes to the Process Control subsystem.

In files relating the process type to other system entities (that is, the Assign/Process Type Relationship File and the EMT/Process Type Relationship File), eight wild card characters can be used (*****) to represent all process types configured in the PTF. Therefore, the wild card character should not be used in this field.

Example:	IAS
Field Length:	1–8 alphanumeric characters
Required Field:	Yes
Default Value:	No default value
Data Name:	PTF.PRIKEY.PRO-TYP

Explanation

Each field description is completed by one or more of the following items of information:

Item	Description
Example	Illustrates a possible entry for the field to further clarify the value or values that can be entered.
Field Length	Specifies the size of the field and the type of characters that can be entered. This length refers to the field and valid values for the file maintenance screen, not the field in the DDLs. Possible values are alphabetic, alphanumeric, hexadecimal, and numeric. The term <i>alphanumeric</i> includes all alphabetic, numeric, and special characters that can be entered from a keyboard without using a control sequence. The term <i>hexadecimal</i> includes all numbers and the letters A through F. When a field value cannot be modified by the operator, the field length is System-protected.
Occurs	Indicates the number of times the field can be displayed on the screen. This information is provided only when the field can be displayed multiple times.
Required Field	Specifies whether a value has to be entered in the field. Possible values are Yes and No. Some fields are required only under certain conditions. In this case, the entry is Yes, followed by the conditions that determine when the field is required.
Default Value	Specifies the value that is automatically placed in the field when the screen is first displayed or when the F8 key is pressed to clear the screen.
Data Name	Provides the DDL name associated with the field appearing on the screen. Data names are included in the documentation to assist in communicating screen and field issues to your technical staff. Note that screen data is not always stored in the BASE24 database as it appears on the screen or as it is described in the field description. If you need information on how screen data is actually stored, consult the DDLs.

Unlabeled Fields on Screens

Angle brackets (< >) indicate an unlabeled field on a screen. An unlabeled field is a field that is present on a screen but is not preceded by an identifying literal label. For the purposes of documenting the field, a label has been assigned and appears inside the angle brackets. A multiple line unlabeled field is displayed as a shaded area and also has a label in angle brackets.

In the field descriptions for the screen, the unlabeled field appears according to its place on the screen and is identified by the same label.

Unlabeled fields are not included in the index by field name; they appear by subject in the main index.

Nucleus Process Terminology Change

There is one process that is the hub of an XPNET node. It manages message queues; controls all lines, stations, and processes in the system; and serves as an interprocess interface. Throughout this manual, this process is referred to as the XPNET process.

In other manuals, you may see references to the Nucleus process as performing these same functions. These two terms (Nucleus process and XPNET process) should be viewed as synonymous. The Nucleus process and the XPNET process perform the same functions and processing. As manuals are reissued, references to the Nucleus process will be replaced with references to the XPNET process.

ACI Worldwide Inc.

Section 1

Introduction

This section provides an introduction to the files and process control functions specific to the integrated transaction services (ITS) kernel environment. It includes introductory information for the ITS kernel basic and relationship file maintenance screens, the ITS kernel process control screens, and the function keys used to access these screens.

ITS Kernel Files and Functions

ITS kernel configuration files are those specific to the integrated transaction services (ITS) kernel environment. ITS kernel process control functions are those functions used to control the object-oriented or procedural software running in the ITS environment. The ITS environment includes the files, functions, and processes used to configure and control ITS processes. The kernel functions as the base of all ITS applications and is composed of a set of objects, files, and screens providing essential services to an application, such as event logging, process management, and memory management. The term procedural refers to software designed and implemented in ACI's traditional process-oriented architecture without the use of objects.

File maintenance and process control screens are presented in this manual in alphabetical order by screen name.

Each file maintenance and process control screen description in this manual includes a screen overview, a screen sample, a discussion of any function keys that differ from the standard BASE24 function keys documented in the ***BASE24 CRT Access Manual***, and a detailed presentation of each field appearing on the screen.

Standard ITS Kernel Files

The following standard ITS kernel basic and relationship files can be accessed using file maintenance screens:

- Assign File (ASGN)
- Assign/Object Relationship File (AORF)
- Assign/Process Type Relationship File (APRF)
- EMT/Object Relationship File (EORF)
- EMT/Process Type Relationship File (EPRF)
- Event Suppression File (ESF)
- Extended Memory Table File (EMT)
- Log Permission Process Partition File (LPF)
- Object File (OBJ)
- Process File (PRO)
- Process/Object Relationship File (PORF)

- Process Type File (PTF)
- Server Class/Process Relationship File (SPRF)
- Server Class Type File (SCT)
- Source Class Map File (SCM)

File maintenance screens for accessing each of these files are described in subsequent sections of this manual. File maintenance screen names are the same as the names of the kernel files to which they provide access. Sections are arranged in alphabetical order by file maintenance screen name and are intermixed with process control screen sections.

Process Control Functions

The following ITS kernel process control functions can be accessed using formatted screens. The names of the screens from which each of these functions is executed are provided in parentheses:

- Building extended memory tables (Configuration Control screen)
- Controlling ITS server processes (Server Control screen)
- Debugging processes (Debug screen)
- Displaying the status of objects (Status screen)
- Reinitializing objects (LCONF Warmboot screen)
- Tracing objects (Trace screen)
- Reallocating individual extended memory tables (EMT Warmboot screen)
- Rereading individual LCONF assigns (Assign Warmboot screen)
- Rereading individual LCONF params (Param Warmboot screen)

Process control functions cannot be performed until after the ITS kernel basic and relationship files have been configured.

Formatted process control screens used to perform each of these functions are described in subsequent sections of this manual. Sections are arranged in alphabetical order by process control screen name and are intermixed with file maintenance screen sections.

File Access

All ITS kernel configuration and process control files accessible to the user are listed on the Kernel (KNL) Product Menu screen. Each operator only sees the files and process control functions available to him or her based on the configuration in the security record. A sample Kernel Product Menu listing all possible files and process control functions is shown below. The order in which the files and functions are displayed for individual users is dependent on the order in which the user received access to the files.

Users can access a file or process control function listed on the menu by placing the cursor beside an individual item and pressing the **F1** key. A more detailed explanation for accessing files and functions is provided in the ***BASE24 CRT Access Manual***.

```
BASE24-ADMN  MENU                LLLL      YY/MM/DD  HH:MM  01 OF 01
              K N L      P R O D U C T M E N U

              AORF          APRF          ASGN          AWB
              CCS           DBG           EMT           EORF
              EPRF          ESF           EWB           LPF
              LWB           OBJ           PORF          PRO
              PTF           PWB           SCM           SCS
              SCT           SPRF          STAT          TRC
```

```
***** BASE24 *****
              FILE DESTINATION:
F1-ENTER DATA  F10-PRINT  F12-HELP  F16-EXIT  SF16-LOGOFF
```


Function Keys

Standard BASE24 function keys are used on the ITS kernel file maintenance and process control screens. These function keys are described in the ***BASE24 CRT Access Manual***. When the use of these keys differs for an ITS kernel file or function, the differences are noted in the section of the manual dealing with the screen grouping to which they apply.

Each screen also has an online Help screen that lists the function keys that apply on the screen. The Help screen is displayed by pressing the **F12** key. Examples of a Help screen for an ITS kernel file maintenance screen and a kernel process control screen are provided at the end of this section.

Default Records

Many of the ITS configuration files are provided with data already preconfigured by ACI prior to installation. This ensures that the files contain accurate data and that they also contain all the information needed by the appropriate applications. The default data in these files should not be altered as it is already used by (or required by) existing applications. This does not preclude you from adding data to these files for your own custom development as long as the default data is not altered or deleted.

The following kernel configuration files contain default data provided by ACI:

- Assign File (ASGN)
- Assign/Object Relationship File (AORF)
- Assign/Process Type Relationship File (APRF)
- Extended Memory Table File (EMT)
- EMT/Object Relationship File (EORF)
- EMT/Process Type Relationship File (EPRF)
- Object File (OBJ)
- Process Type File (PTF)

The preconfigured default data is listed at the end of each applicable section in this manual.

Customer-Configured Data

Data in several files must be configured during or after installation to contain the processing values required for your specific processing requirements.

The Source Class Map File (SCM) must contain an entry for each source class of a process or station from which messages can originate. The entry identifies the message handler object used for each source class. For systems running under XPNET 3.0, the source class must match the value of the SOURCECLASS attribute for processes and stations from which messages can originate.

For systems running under XPNET 2.1, the source class must match the value of the Sourceclass element for processes and stations from which messages can originate. For systems running under XPNET 2.0, the source class must match the value of the Class element for processes and stations from which messages can originate.

The following configuration files documented in this manual typically require some update during or after your ITS system installation. For detailed step-by-step procedures for configuring these files, refer to the ***BASE24 ITS Kernel Configuration Manual***.

- Process/Object Relationship File (PORF)
- Process File (PRO)
- Server Class Type File (SCT)
- Server Class/Process Relationship File (SPRF)

Data Entry Hierarchy

Data in the ITS environment is configured in two types of kernel files. Basic files establish each of the fundamental components of an ITS configuration. Relationship files establish each of the relationships that exist among these fundamental components for process control purposes. Relationship files cannot be configured until the data for the respective basic files they relate has been configured.

Basic Files

The following kernel files in the ITS environment are considered to be basic files. Basic files establish the fundamental components of an ITS environment. These files must be configured before information in them can be related in relationship files.

- Assign File (ASGN)
- Extended Memory Table File (EMT)
- Event Suppression File (ESF)
- Log Permission Process Partition File (LPF)
- Object File (OBJ)
- Process File (PRO)
- Process Type File (PTF)
- Server Class Type File (SCT)
- Source Class Map File (SCM)

Basic file maintenance screens are documented in alphabetical order within sections 2 through 25 and are intermixed with relationship file screens and process control screens.

Relationship Files

Relationship files establish associations between components in the ITS environment. Before you can establish a relationship between two components using the relationship files, you must define the components in the basic files. The following ITS kernel files are considered to be relationship files.

- Assign/Object Relationship File (AORF)
- Assign/Process Type Relationship File (APRF)
- Extended Memory Table/Object Relationship File (EORF)
- Extended Memory Table/Process Type Relationship File (EPRF)
- Process/Object Relationship File (PORF)
- Server Class Type/Process Relationship File (SPRF)

Relationship file maintenance screens are documented in alphabetical order in sections 2 through 25 and are intermixed with basic file maintenance screens and process control screens.

Note: In addition to the files above, the Destination File (DEST) is also a relationship file. The DEST is used with an Integrated Endpoint process to define message routing. For more information on the DEST, see the ***BASE24 IEP Configuration Manual***.

File Relationships and Dependencies

File relationships and dependencies exist among the ITS kernel basic and relationship files. Data must be entered according to these relationships and dependencies. For example, the appropriate records must exist in both the Process Type File (PTF) and the Assign File (ASGN), before process types and assigns can be related in the Assign/Process Type Relationship File (APRF). In another example, the appropriate records must already exist in the Process Type File (PTF), before processes of a particular type can be added to the Process File (PRO).

The table on the following page identifies the dependencies for ITS kernel files. Files that do not have any dependencies (that is, that do not require corresponding data to be previously entered in another file) are not included in the table. The files that do not rely on data in other files include the ASGN, EMT, ESF, OBJ, PTF, and SCT. Data should be entered in these files first, then in the files identified in the table.

Note: The ESF and LPF should not be configured until after you have some experience running your ITS software in a production environment. See the *BASE24 ITS Kernel Configuration Manual* for detailed instructions on configuring event suppression using the ESF and LPF.

This File	Requires Data in These Files
PRO	PTF
SCM	OBJ
LPF	PTF and PRO
APRF	PTF and ASGN
AORF	ASGN and OBJ
EORF	EMT and OBJ
EPRF	EMT and PTF
PORF	PTF, PRO, and OBJ
SPRF	PTF, PRO, and SCT

Process Control Function Dependencies

All basic and relationship files should be configured prior to using any of the ITS kernel process control screens. Many of the network operations performed using the process control screens are automated according to the configured database. For example, once you have selected an assign to be reread on the Assign Warmboot (AWB) screen, the Process Control subsystem reads various ITS configuration files and automatically determines which objects use the selected assign. A list of all objects that use the assign is displayed on AWB screen 2. You can then select objects from this list to receive the ASSIGN WARMBOOT command. All processes that contain those objects selected will then reread the specified assign.

The table below identifies the dependencies for ITS process control functions. The acronym for the process control screen from which the function is performed is provided in parentheses. All process control functions are dependent on the preexistence of data in one or more configuration files.

This Process Control Function	Requires Data in These Files
Assign Warmboot (AWB)	AORF, APRF, ASGN, PORF, PRO, SCT, and SPRF
EMT Build (CCS)	EMT
Debug (DBG)	PRO, SCT, and SPRF
EMT Warmboot (EWB)	EMT, EORF, EPRF, PORF, PRO, SCT, and SPRF
LCONF Warmboot (LWB)	OBJ, PORF, PRO, SCT, and SPRF
Param Warmboot (PWB)	OBJ, PORF, PRO, SCT, and SPRF
Status Command (STAT)	OBJ, PORF, PRO, SCT, and SPRF
Server Control (SCS)	OBJ, PORF, PRO, SCT, and SPRF
Object Trace (TRC)	OBJ, PORF, PRO, SCT, and SPRF

Help Screens

BASE24 products provide online Help screens that list the function keys that apply on each screen. Help screens are displayed by pressing the **F12** key.

Help screens are not available for the Logon screen or the Virtual menu. However, the function keys that are available on these screens are listed at the bottom of the screens.

Examples of Help screens for a kernel file maintenance screen and a process control screen are shown below and on the following page.

The following is the Help screen for OBJ screen 1.

```
BASE24-KNL   OBJECT FILE           LLLL   YY/MM/DD  HH:MM  01 OF 01

                OBJECT NAME:
STANDARD FILE MAINTENANCE FUNCTION FOR KEYS 6520/6530 (3270)
      F1 (ENTER) - VALIDATE DATA
      F2 (PF2)  - READ RECORD
      F3 (PF3)  - ADD RECORD
      F4 (PF4)  - DELETE RECORD
      F5 (PF5)  - UPDATE RECORD
      F6 (PF6)  - READ NEXT RECORD
      F7 (PF7)  - GOTO NEW PAGE
      F8 (PF8)  - CLEAR SCREEN
      F9 (PF9)  - NEXT SCREEN
      F10 (PF10) - PRINT SCREEN
      F11 (PF11) - PREVIOUS SCREEN
      F12 (PF12) - DISPLAY HELP SCREEN
      F13 (PF13) - CHANGE CURRENT LOGICAL NET
      F16 (PF16) - EXIT OR GOTO FILE
      SF16 (PA2) - LOGOFF

***** BASE24 *****
NEW PAGE:           FILE DESTINATION:           NEW LOGICAL NETWORK ID:
ANY FUNCTION KEY EXCEPT SF9-SF16, F10, OR F16 RETURNS. PF10 PRINTS AND RETURNS
```


The following is the Help screen for AWB screen 1.

BASE24-KNL ASSIGN WARMBOOT LLLL YY/MM/DD HH:MM 01 OF 02

STANDARD FILE MAINTENANCE FUNCTION FOR KEYS 6520/6530 (3270)

F1 (ENTER) - SEND COMMAND
SF7 (PF22) - VIEW PREVIOUS PAGE OF MULTI-PAGE ENTRY
SF8 (PF23) - VIEW NEXT PAGE OF MULTI-PAGE ENTRY
F10 (PF10) - PRINT SCREEN
F12 (PF12) - HELP
F16 (PF16) - EXIT/GO TO NEW DESTINATION/CHANGE LOGICAL NETWORK
SF16 (PA2) - LOGOFF

***** BASE24 *****
NEW PAGE: FILE DESTINATION: NEW LOGICAL NETWORK ID:
ANY FUNCTION KEY EXCEPT SF9-SF16, F10, OR F16 RETURNS. PF10 PRINTS AND RETURNS

Object and Extended Memory Table Abbreviations

Object and extended memory table name abbreviations are used extensively on ITS kernel configuration and process control screens. The following cross-reference tables are provided for your convenience.

Object Name Abbreviations

The following table cross-references object abbreviations to complete object names.

Object Abbreviation	Complete Object Name
ACAC	Account Access object
ACCT	Account object
ACPC	Authorization Combined Positive Customer object
ACU	Authorization Combined Utility object
ANS	Application Network Services object
ASAC	Authorization Security Access object
CLAS	Class object
CSAC	Customer Access object
CSI	Customer Service Interface object
CTXM	Context Manager object
CUST	Customer object
DEST	Destination object
EOPP	End-of-Period Process object
EPCS	Endpoint Class Handler Services object
EVT	Event object
FC	Fraud Control object

Object Abbreviation	Complete Object Name
FI	Financial Institution object
FIAC	Financial Institution Access object
GENE	Generic Endpoint Class Handler object
HIST	Transaction History object
IMGR	Interface Manager object
IMS	Interface Master Services object
IMTT	Internal Message Translation Interface object
LOGP	Log Permission object
MC	Multiple Currency object
MISO	Message ISO object
PCTL	Process Control object
RTE	Router object
TRC	Trace object
TSEC	Transaction Security object
TXN	Transaction object
VRUE	Voice Response Unit Endpoint Class Handler object
VRUM	Voice Response Unit Master object
VRUT	Voice Response Unit Interface object
XPN	XPNET Interface object

Extended Memory Table Name Abbreviations

The following table cross-references extended memory table name abbreviations to complete extended memory table names.

EMT Abbreviation	Complete EMT Name
CPFEMT	Card Prefix File extended memory table
CUSTEMT	Customer extended memory table
DESTEMT	Destination File extended memory table
ESEMT	Event suppression extended memory table
FPFEMT	Fraud Parameters File extended memory table
IDFEMT	Institution Definition File extended memory table
IMTEMT	Internal Message Translation File extended memory table
INTFEMT	Interface extended memory table
IRTEEMT	Institution routing extended memory table
ISECEMT	Interface Security File extended memory table
KEYAEMT	Key Authorization File extended memory table
SCMEMT	Source Class Map File extended memory table
SSMEMT	Source symbolic map extended memory table
THFEMT	Transaction history file extended memory table
TLFEMT	Transaction log file extended memory table

Section 2

Assign File (ASGN)

The Assign File (ASGN) identifies Logical Network Configuration File (LCONF) assigns that can be reread using the ASSIGN WARMBOOT command. If an assign is not supported by the ASSIGN WARMBOOT command, it does not need to be defined in this file and can be reread only when a process is initialized or reinitialized.

The records in this file are used to build the assign entries that appear on Assign Warmboot screen 1. There is one ASGN record for each assign that can be reread.

The ASGN is delivered with all assigns required for kernel, integrated endpoint, and core objects for ACI-developed applications preconfigured. These default records are described at the end of this section.

When you first enter Assign Warmboot screen 1, the Assign Warmboot Server process reads the ASGN and displays each record found. You can then select an assign to be reread from this list.

For each assign defined in the ASGN, a corresponding assign record should exist in the Logical Network Configuration File (LCONF); however, you can add assigns to this file before they are added to the LCONF since no cross-editing checks are performed.

The ASGN is secured under the Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

Records associating an assign to an object and a process type in the Assign/Object Relationship File (AORF) and Assign/Process Type Relationship File (APRF), respectively, must be deleted before you can delete the associated ASGN record.

The key to ASGN records is the ASSIGN NAME field.

Screen 1

ASGN screen 1 allows you to identify the assigns that can be reread in the ITS environment using the ASSIGN WARMBOOT command. ASGN screen 1 is shown below, followed by a description of each field on the screen.

```
BASE24-KNL  ASSIGN FILE          LLLL      YY/MM/DD  HH:MM  01 OF 01

                ASSIGN NAME:

                ASSIGN DESCRIPTION:


***** BASE24 *****
NEW PAGE:      FILE DESTINATION:      NEW LOGICAL NETWORK ID:
                F12-HELP
```

ASSIGN NAME — The name of an assign as defined in the Logical Network Configuration File (LCONF).

Field Length: 1–32 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: ASGN.PRIKEY.ASGN-NAM

ASSIGN DESCRIPTION — A description of the assign. The description entered in this field appears on the AORF, APRF, and Assign Warmboot (AWB) screens.

Field Length: 1–20 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: ASGN.ASGN-DESCR

Default Records

The following preconfigured default records for the ASGN are provided by ACI. These default records are reserved for use by ACI and its customers and should not be altered for custom development.

ASSIGN NAME	ASSIGN DESCRIPTION
ELF	EXCEPTION LOG FILE
LPP	LOG PERMISSION PRO
SECURE-DEVICE	SECURITY DEVICE
THF	TRANS HISTORY FILE
TLF	TRANSACTION LOG FILE

ACI Worldwide Inc.

Section 3

Assign/Object Relationship File (AORF)

The Assign/Object Relationship File (AORF) establishes relationships between Logical Network Configuration File (LCONF) assigns and ITS objects that reread these assigns using the ASSIGN WARMBOOT command. The records in this file are used to build the entries that appear on Assign Warmboot screen 2. There is one AORF record for each combination of assign and object that can reread the assign.

Note: Only assigns that can be reread using the ASSIGN WARMBOOT command need to be related in this file. Some assigns may exist that cannot be reread using this command and are only read when a process is initialized or reinitialized. The AORF is delivered with all assign and object relationships required for kernel objects and ACI-developed applications preconfigured. This default data is described at the end of this section.

When you select an assign to be reread from the list of assigns on Assign Warmboot screen 1, the Assign Warmboot Server process uses the selected assign as a key to read the AORF. The Assign Warmboot Server process retrieves the name of each object associated with the assign and displays the object names on Assign Warmboot (AWB) screen 2. You can then identify the specific objects that can receive the ASSIGN WARMBOOT command to reread the assign.

The assigns named in this file must be defined in the LCONF. Before you can add records to the AORF, the assign name must be defined in the Assign File (ASGN) and the object name must be defined in the Object File (OBJ).

Records in the AORF must be deleted before you can delete the associated ASGN and OBJ records.

The AORF is secured under the Tandem group level security. Only authorized operators can access the file or start programs that access the file.

The key to AORF records is a combination of the ASSIGN NAME and the OBJECT NAME fields.

Screen 1 Function Keys

The use of one function key on AORF screen 1 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The **F5** key is not valid on this screen. To update an AORF record, you must delete the existing record and then add a new record with the updated information.

Screen 1

AORF screen 1 allows you to define the relationships in your ITS environment between assigns that can be reread using the ASSIGN WARMBOOT command and objects. AORF screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
ASSIGN/OBJECT RELATION
LLLL
YY/MM/DD
HH:MM
01 OF 01

ASSIGN NAME:
OBJECT NAME:

(<assign description>)
(<object description>)

BASE24

NEW PAGE:
FILE DESTINATION:
NEW LOGICAL NETWORK ID:

F12-HELP

ASSIGN NAME — The name of the assign as defined in the Logical Network Configuration File (LCONF) and the Assign File (ASGN).

Field Length:
1–32 alphanumeric characters

Required Field:
Yes

Default Value:
No default value

Data Name:
AORF.PRIKEY.ASGN-NAM

<assign description> — A description of the assign as defined in the ASGN. This description is retrieved from the ASGN after a valid assign name is entered in the ASSIGN NAME field.

Field Length:
System-protected

Data Name:
ASGN.ASGN-DESCR

OBJECT NAME — The name of the object which uses this file assign. The entry in this field must match a name for an object already defined in the Object File (OBJ).

Field Length: 1–4 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: AORF.PRIKEY.ALTKEY.OBJ-NAM

<object description> — The description of the object from the OBJ. This description is retrieved from the OBJ after a valid object name is entered in the OBJECT NAME field.

Field Length: System-protected
Data Name: OBJ.OBJ-DESCR

Default Records

The following preconfigured default records for the AORF are provided by ACI. These default records are reserved for use by ACI and its customers and should not be altered for custom development.

ASSIGN NAME	OBJECT NAME
ELF	EOPP
LPP	EVT
SECURE-DEVICE	TSEC
THF	HIST
TLF	EOPP

ACI Worldwide Inc.

Section 4

Assign/Process Type Relationship File (APRF)

The Assign/Process Type Relationship File (APRF) establishes relationships between Logical Network Configuration File (LCONF) assigns and process types. The relationship indicates that the process type uses the assign and that processes of that type can reread the specific assign using the ASSIGN WARMBOOT command. The records in this file are used to validate the process type during assign warmboot processing. There is one APRF record for each combination of assign and process type that can reread the assign.

Note: Only assigns that can be reread using the Assign Warmboot screen need to be related in this file. Some assigns may exist that cannot be reread and are only read when a process is initialized or reinitialized.

When you select the objects for which an assign is to be reread on Assign Warmboot (AWB) screen 2, the Assign Warmboot Server process reads the APRF to determine which process types allow the assign to be reread. The server process uses the process type of processes containing the selected objects to read the APRF. If an object resides in a process with a process type that is not configured in the APRF, the ASSIGN WARMBOOT command is not sent to that process.

The assigns named in this file must be defined in the LCONF. Before you can add records to the APRF, the assign name must be defined in the Assign File (ASGN) and the process type must be defined in the Process Type File (PTF).

The APRF is secured under the Tandem group level security. Only authorized operators can access the file or start programs that access the file.

The key to APRF records is a combination of the ASSIGN NAME and the PROCESS TYPE fields.

Screen 1 Function Keys

The use of one function key on APRF screen 1 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The **F5** key is not valid on this screen. To update an APRF record, you must delete the existing record and then add a new record with the updated information.

Screen 1

APRF screen 1 allows you to define the relationships between assigns that can be reread using the ASSIGN WARMBOOT command and process types in your ITS environment. APRF screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
ASSIGN/PRO TYPE RELATION LLLL
YY/MM/DD HH:MM 01 OF 01

ASSIGN NAME:
PROCESS TYPE:

(<assign description>)
(<process type descrip>)

***** BASE24 *****

NEW PAGE:
FILE DESTINATION:
NEW LOGICAL NETWORK ID:
F12-HELP

ASSIGN NAME — The name of an assign as defined in the Logical Network Configuration File (LCONF) and in the Assign File (ASGN).

Field Length:
Required Field:
Default Value:
Data Name:

1–32 alphanumeric characters
Yes
No default value
APRF.PRIKEY.ASGN-NAM

<assign description> — A description of the assign as defined in the ASGN. This description is retrieved from the ASGN after a valid assign name is entered in the ASSIGN NAME field.

Field Length:
Data Name:

System-protected
ASGN.ASGN-DESCR

PROCESS TYPE — The acronym indicating the type of process to which this file assign is related. The entry in this field must either match an acronym for a process type already defined in the Process Type File (PTF) or be filled with asterisks. All asterisks in this field indicates that an assign can be reread by all process types using the ASSIGN WARMBOOT command.

Field Length: 1–8 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: APRF.PRIKEY.ALTKEY.PRO-TYP

<process type descrip> — A description of the process type retrieved from the PTF. This description is retrieved from the PTF after a valid process type is entered in the PROCESS TYPE field. When all asterisks are entered in the PROCESS TYPE field, the value WILDCARD is displayed in this field.

Field Length: System-protected
Data Name: PTF.PRO-TYPE-DESCR

Default Records

The following preconfigured default records for the APRF are provided by ACI. These default records are reserved for use by ACI and its customers and should not be altered for custom development.

ASSIGN NAME	PROCESS TYPE
ELF	EOPP
LPP	*****
SECURE-DEVICE	IAS
SECURE-DEVICE	IMP
THF	IAS
TLF	EOPP

ACI Worldwide Inc.

Section 5

Assign Warmboot (AWB) Screen

The Assign Warmboot (AWB) screen allows you to select specific LCONF assigns to be reread by an object. An assign warmboot should be done any time a change is made to an assign and the assign can be reread by the object using the ASSIGN WARMBOOT command.

LCONF assigns used by kernel objects are listed by object in the *BASE24 ITS Kernel Objects Reference Manual*. Assigns used by integrated endpoint objects are listed by object in the *BASE24 IEP Objects Reference Manual*. Assigns used by core objects are listed in the application-specific transaction processing manual. Each listing indicates whether the assign can be reread using the ASSIGN WARMBOOT command. Individual LCONF assign descriptions are provided in the *BASE24 Logical Network Configuration File Manual*.

In general, any assign that cannot be reread without the rebuilding of internal tables or a secondary frame, or that is dependent on another assign or param, cannot be reread using the ASSIGN WARMBOOT command. The intent of the ASSIGN WARMBOOT command is to simply swap the old value for the assign with the new value, without performing any reinitialization processing.

Only assigns that can be reread using the ASSIGN WARMBOOT command should be configured in the Assign File (ASGN). Assigns from this file are displayed on AWB screen 1. A description of the ASGN screen is provided elsewhere in this manual.

If an assign in the LCONF has been changed and cannot be reread by an object using the ASSIGN WARMBOOT command, use the LCONF Warmboot screen to reread all LCONF assigns for the objects affected.

A changed assign is not used in processing until the assign is reread.

The Assign Warmboot Server process uses the following files to validate and automate the processing of an ASSIGN WARMBOOT command. Records must be properly defined in these files before the Assign Warmboot screen can be used. Descriptions for each of these files can be found elsewhere in this manual.

- Assign/Object Relationship File (AORF)
- Assign/Process Type Relationship File (APRF)
- Assign File (ASGN)
- Process/Object Relationship File (PORF)
- Process File (PRO)
- Server Class Type/Process Relationship File (SPRF)
- Server Class Type File (SCT)

The AWB is secured under the Tandem group level security so that only authorized network operators can access the Assign Warmboot function.

The following screens are used to process Assign Warmboot requests:

- AWB screen 1 allows you to select the assigns to be warmbooted.
- AWB screen 2 allows you to select the objects to receive the ASSIGN WARMBOOT command.

Screen 1 Function Keys

The use of the following keys on AWB screen 1 varies from the standard function keys explained in the *BASE24 CRT Access Manual*. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects an assign to be reread and displays AWB screen 2. You must identify the assign to be reread by placing the value Y in the SELECT field on the same line as the assign prior to pressing this function key.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of assign entries if there are more entries than will fit on one screen.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of assign entries if there are more entries than will fit on one screen.

Screen 1

AWB screen 1 allows you to select an LCONF assign to be reread. When you access this screen, all assigns currently defined in the ASGN are displayed. You select an assign to be reread by placing the value Y in the SELECT field next to the assign name and pressing the **F1** function key.

All assigns configured to the ASGN file are displayed in a list on this screen. You can select only one assign at a time from this list to be reread. Once you have selected an assign, AWB screen 2 is displayed, from which you can select the objects to reread the assign.

AWB screen 1 is shown below, followed by a description of each field on the screen.

```
BASE24-KNL  ASSIGN WARMBOOT          LLLL          YY/MM/DD  HH:MM  01 OF 02
```

```
SELECT THE ASSIGN NAME
```

```
SELECT  ASSIGN NAME
```

```

N      ELF
N      LPP
N      SECURE-DEVICE
N      THF
N      TLF

```

```
***** BASE24 *****
```

```
FILE DESTINATION:          NEW LOGICAL NETWORK ID:
```

```
F1-ENTER  SF7-PREVIOUS PAGE  SF8-NEXT PAGE  F12-HELP
```

SELECT — A flag indicating whether the assign displayed on the same line of the screen is to be reread. Only one assign can be selected at a time. Valid values are as follows:

Y = Yes, reread this assign.

N = No, do not reread this assign.

Field Length: 1 alphabetic character
Occurs: Up to 12 times
Required Field: Yes
Default Value: N
Data Name: Not applicable

ASSIGN NAME — The name of an LCONF assign that can be reread using the ASSIGN WARMBOOT command. All assigns currently defined in the ASGN are displayed in this field.

Field Length: System-protected
Occurs: Up to 12 times
Data Name: Not applicable

Screen 2 Function Keys

The use of the following keys on AWB screen 2 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects the objects to reread the specified assign. You must identify the objects to reread the assign by placing the value Y in the SELECT fields on the same line as each object prior to pressing this function key. You can select objects on multiple pages prior to pressing the F1 key.
F15	Return to Page 1 — Displays AWB screen 1.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of object entries if more than 12 objects use the specified assign.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of object entries if more than 12 objects use the specified assign.

Screen 2

AWB screen 2 allows you to select objects that should reread the specified assign. All objects that are related to the assign selected on AWB screen 1 in the Assign/Object Relationship File (AORF) are displayed on this screen. If more than one page of object entries exist, you can view more object selections by pressing the **Shift-F7** and **Shift-F8** keys.

You select objects by placing the value Y in the SELECT field on the same line as the object for which the specified assign is to be reread. All processes containing the selected objects that have a process type configured in the Assign/Process Type Relationship File (APRF) then automatically receive the ASSIGN WARMBOOT command when you press the **F1** key. Each object receiving the ASSIGN WARMBOOT command verifies that the name is correct and that it applies to the object before the assign is reread.

When the ASSIGN WARMBOOT command is successfully sent, the message “Command Processed Successfully” is displayed on the screen.

AWB screen 2 is shown below, followed by a description of each field on the screen.

```
BASE24-KNL    ASSIGN WARMBOOT          LLLL          YY/MM/DD  HH:MM  02 OF 02
```

```
THE OBJECTS LISTED BELOW SHARE THE ASSIGN CHOSEN.  SELECT THOSE
OBJECTS WHICH REQUIRE WARMBOOTING.
```

```
SELECT      OBJECT DESCRIPTION
```

```
    N          EVT
```

```
***** BASE24 *****
```

```
FILE DESTINATION:          NEW LOGICAL NETWORK ID:
```

```
F1-ENTER  F15-RETURN TO PAGE 1  SF7-PREVIOUS PAGE  SF8-NEXT PAGE  F12-HELP
```

SELECT — A flag indicating whether the object displayed on the same line of the screen is to reread the specified assign. Valid values are as follows:

Y = Yes, the object should reread the specified assign.

N = No, the object should not reread the specified assign.

Field Length: 1 alphabetic character

Occurs: Up to 12 times

Required Field: Yes

Default Value: N

Data Name: Not applicable

OBJECT DESCRIPTION — The text description of an object that uses the specified assign. All objects currently related to the assign in an AORF record are displayed.

Field Length: System-protected

Occurs: Up to 12 times

Data Name: Not applicable

Section 6

Configuration Control (CCS) Screen

The Configuration Control (CCS) screens provide a mechanism to build or rebuild shared extended memory tables as needed. When a shared extended memory table is selected from the list on Configuration Control screen 2, the Configuration Control Server process starts the Extended Memory Table Build utility by sending it a startup command. When the Extended Memory Table Build utility receives this command, it rebuilds the specified extended memory table. A new shared extended memory table is not used in processing, however, until all processes that access the table are informed of the change and reallocate the table.

Note: The Extended Memory Table Build utility is always run at a low process identification number (PIN) when started from the CCS screens. The utility can run at a low or high PIN when started from a TACL prompt. Instructions for starting the utility from a TACL prompt are provided in the *BASE24 ITS Kernel Configuration Manual*.

The Configuration Control Server process retrieves the list of shared extended memory tables for screen 2 from the Extended Memory Table File (EMT). Therefore, before this screen can be used to build an extended memory table, a record for the extended memory table must be defined in the EMT.

Warning: The source files or tables from which the selected extended memory tables are built will be locked until the Extended Memory Table Build utility completes processing.

When the Configuration Control command is successfully sent, the message “Command Processed Successfully” is displayed on the screen.

The CCS screens are secured under the Tandem group level security so that only authorized network operators can access the Extended Memory Table Build utility.

The following screens are used to process Extended Memory Table Build requests:

- CCS screen 1 allows you to specify the runtime parameters for the Extended Memory Table Build utility.
- CCS screen 2 allows you to select which extended memory tables are to be rebuilt during the run.

Instructions are provided for reallocating an extended memory table after it has been built or rebuilt from the CCS.

Reallocating a Newly Built or Rebuilt Extended Memory Table

Once a shared extended memory table is built or rebuilt from the CCS, it will not be used until the application that uses the extended memory table has been informed of the change and reallocates the table. The procedures for informing processes vary according to the type of application process (that is, object-oriented or procedural) that uses the table and the environment (for example, XPNET or Pathway) in which the process runs.

Object-Oriented Processes

Extended memory tables are reread by object-oriented processes using the EMT WARMBOOT command and the LCONF WARMBOOT command. The EMT WARMBOOT command is entered from the EMT Warmboot (EWB) screen and allows individual objects to reallocate an individual extended memory table. When an object receives the EMT WARMBOOT command, it reallocates the table if it is shared and rebuilds the table if it is not shared. This command is only allowed if the EMT WARMBOOT ALLOWED field on EMT screen 1 is set to a value of Y for the extended memory table. If not, you must use the LCONF WARMBOOT command. The LCONF WARMBOOT command is entered from the LCONF Warmboot screen (LWB) and causes the entire process to reinitialize, including the allocation of extended memory tables.

The EMT WARMBOOT command causes an object to validate the name of the extended memory table, allocate the new extended memory table, deallocate the old extended memory table, and replace the extended memory table segment ID in the frame. If more complicated processing is required for an extended memory table (for example, building a secondary frame), then you must use the LCONF WARMBOOT command to reallocate the table.

Procedural Processes

Procedural processes that use extended memory tables reallocate extended memory tables using the warmboot facilities provided for the environment in which they run—Pathway or XPNET. Use of these facilities is described in the following paragraphs.

Pathway Environment

To cause a procedural server process running under control of the Pathway process to reallocate an extended memory table, perform the following:

1. Start a Pathcom session from a TACL prompt.
`TACL> PATHCOM pathmon-process-name`
2. From the Pathcom prompt, freeze the server process.
`= FREEZE SERVER server-name`
3. Stop the server process.
`= STOP SERVER server-name`
4. Thaw the server process.
`= THAW SERVER server-name`
5. Start the server process.
`= START SERVER server-name`

When the server process is started it will reinitialize, causing the server process to reallocate the extended memory table. If the server process was configured as a dynamic server, you will receive an error message which can be ignored.

For more information on PATHCOM commands, refer to the *NonStop TS/MP System Management Manual*.

XPNET Environment

To cause a procedural process running under control of the XPNET process to reallocate an extended memory table, perform the following:

1. Access a network control facility (for example, NCPcom or the Network Control Supervisor screen).
2. Stop and start the process or warmboot the process.
 - a. To stop and start a process, enter the following commands from a network control facility.

```
STOP PROCESS sym-proc-name  
START PROCESS sym-proc-name
```

For detailed syntax information for these commands, refer to the *NET24-XPNET Network Control Command Reference Manual*.

- b. To warmboot a process, enter the appropriate DELIVER PROCESS WARMBOOT (9501) command for the target process. The exact syntax of the command may vary according to the type of process for which the command is intended. For detailed information on this command, refer to the ***BASE24 Text Command Reference Manual***.

Screen 1 Function Keys

The use of the following key on CCS screen 1 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The use of this function key is explained below.

The first column of information shows the BASE24 key. The second column describes the function that can be accomplished with this key.

Key	Description
F1	Enter — Enters the runtime parameters and displays CCS screen 2.

Screen 1

CCS screen 1 allows operators to specify runtime information for the Extended Memory Table Build utility. Through this screen, the operator enters the volume and subvolume location of the utility, a process pair directory (PPD) name for the utility while it runs, the execution priority at which the utility will run, the CPU in which the utility will run, and the spooler location for a processing summary report. Once this information is entered, press the **F1** key to display screen 2. From CCS screen 2, you can select the extended memory tables to be built.

A high-level processing description of the Extended Memory Table Build utility is provided in the ***BASE24 ITS Kernel Objects Reference Manual***.

CCS screen 1 is shown below, followed by a description of each field on the screen.

```

BASE24-KNL   CONFIGURATION CONTROL   LLLL           YY/MM/DD  HH:MM  01 OF 02

                W A R N I N G !

Execution of this screen will initiate the building of a new extended
memory segment for use by the application servers.  Additionally, the
specified files will be locked until the program completes.

PROGRAM LOCATION
(SYS/VOL/SUBVOL):

        PROCESS NAME: $EMTB
          PRIORITY: 139
            CPU: 00

        REPORT LOCATION: $S.#EMTBLD

***** BASE24 *****
                FILE DESTINATION:      NEW LOGICAL NETWORK ID:
F1-ENTER          F12-HELP

```

PROGRAM LOCATION — The location of the executable Extended Memory Table Build utility. The location should include the volume and subvolume for the Extended Memory Table Build utility. The system name is optional. The system name defaults to the name of the system where the Integrated Transaction Services Control Server process is running. The Configuration Control Server process uses the location specified in this field with a file name of EMTBLO to identify the program to start.

Example: \B24.\$DATA.OK13OBJ
Field Length: 1–25 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: Not applicable

PROCESS NAME — The Tandem process pair directory (PPD) name to be given to the Extended Memory Table Build utility while it is running.

Field Length: 2–6 alphanumeric characters
Required Field: Yes
Default Value: \$EMTB
Data Name: Not applicable

PRIORITY — The operating system execution priority at which the Extended Memory Table Build utility is to run. The utility should run at a lower execution priority than online transaction processing. Valid values range from 1 to 180.

Field Length: 1–3 numeric characters
Required Field: Yes, if a value is entered in the CPU field
Default Value: The current Integrated Transaction Services Control Server process priority minus 1
Data Name: Not applicable

CPU — The number of the CPU in which the Extended Memory Table Build utility is to run. Valid values range from 00 to 15.

Field Length: 2 numeric characters
Required Field: Yes, if a value is entered in the PRIORITY field
Default Value: The CPU in which the Integrated Transaction Services Control Server process is running
Data Name: Not applicable

REPORT LOCATION — The spooler or terminal location for the Extended Memory Table Build Report. This is a report summarizing the processing performed by the Extended Memory Table Build utility. A sample Extended Memory Table Build Report is provided in the ***BASE24 ITS Kernel Configuration Manual***.

Any processing errors encountered in running the Extended Memory Table Build utility are written to the processing summary report.

Field Length:	2–11 alphanumeric characters
Required Field:	Yes
Default Value:	\$\$.#EMTBLD
Data Name:	Not applicable

Screen 2 Function Keys

The use of the following keys on CCS screen 2 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Rebuilds the selected extended memory tables. You must identify the extended memory tables to be rebuilt by placing the value Y in the SELECT field on the same line as the extended memory table prior to pressing this function key. You can select extended memory tables on multiple pages prior to pressing the F1 key.
F15	Return to Page 1 — Displays CCS screen 1.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of extended memory table entries if there are more entries than will fit on one screen.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of extended memory table entries if there are more entries than will fit on one screen.

Screen 2

CCS screen 2 allows you to select which extended memory tables are to be built during this run. When you press the **F1** key on CCS screen 1, those extended memory tables with the SUPPORTED BY EMT BUILD FLAG field set to the value Y in the EMT are displayed on this screen. If more than one page of extended memory table entries exist, you can view more extended memory table selections by pressing the **Shift-F7** and **Shift-F8** keys.

Extended memory tables are selected to be rebuilt by placing the value Y next to the name of each extended memory table to be rebuilt and pressing the **F1** key. When the Extended Memory Table Build utility is started, the message “Current EMT Build Process State: Runnable” is displayed at the bottom of the screen.

The extended memory tables are built in the memory of the processor specified by the EMT-SWAPVOL param in the LCONF for the Extended Memory Table Build utility.

CCS screen 2 is shown below, followed by a description of each field appearing on the screen.

```
BASE24-KNL  CONFIGURATION CONTROL  LLLL  YY/MM/DD  HH:MM  02 OF 02
```

```
SELECT THE EMTS WHICH NEED TO BE REBUILT.
```

```
SELECT  EMT NAME
```

```

N      CPFEMT
N      CUSTEMT
N      DESTEMT
N      ESEMT
N      FPFEMT
N      IDFEMT
N      IMTEMT
N      INTFEMT
N      IRTEMT
N      KEYAEMT
N      SCMEMT
N      SSMEMT

```

```
***** BASE24 *****
```

```
FILE DESTINATION:  NEW LOGICAL NETWORK ID:
```

```
F1-ENTER  F15-RETURN TO PAGE 1  SF7-PREVIOUS PAGE SF8-NEXT PAGE  F12-HELP
```

SELECT — A flag indicating whether the extended memory table displayed on the same line of the screen is to be built in processor memory. Valid values are as follows:

Y = Yes, build this extended memory table.

N = No, do not build this extended memory table.

Field Length: 1 alphabetic character

Occurs: Up to 12 times

Required Field: Yes

Default Value: N

Data Name: Not applicable

EMT NAME — The acronym of an extended memory table. All extended memory tables specified in the EMT with the SUPPORTED BY EMT BUILD field set to the value Y are displayed.

Field Length: System-protected

Occurs: Up to 12 times

Data Name: Not applicable

Section 7

Debug (DBG) Screen

The Debug (DBG) screen allows you to run a particular ITS environment process under control of the Tandem debugging tools, Inspect or Debug, for debugging purposes. When a process receives the DEBUG command, it is immediately placed under control of the Tandem Inspect or Debug product. An Inspect or Debug prompt is then displayed on the process's home terminal, which cannot be a pseudoterminal (that is, \$SUDO) since this would cause the Inspect or Debug session to terminate. When processes are bound, you can specify whether the DEBUG command causes a process to be brought up under Inspect or Debug. All ACI processes developed for the ITS environment are bound to start up in Inspect rather than Debug.

The Debug Server process uses the following files to deliver the DEBUG command to the proper processes:

- Process File (PRO)
- Server Class Type/Process Relationship File (SPRF)
- Server Class Type File (SCT)

When a process receives the DEBUG command, the Application Network Services object sends the Debug member function to the Process Control object to begin a Debug session. The Application Network Services object then places all objects in the process under control of Inspect or Debug.

Warning: Debugging is typically only performed in a development environment. It should not be used in a production environment unless absolutely necessary. Running Debug or Inspect on production processes may cause severe degradation in system performance.

Records must be properly configured in the Process File (PRO), Server Class Type File (SCT), and the Server Class Type/Process Relationship File (SPRF), before the Debug facility will work.

The DBG screen is secured under Tandem group level security so that only authorized network operators can access this debugging function.

For detailed information on using Inspect, refer to the Tandem ***INSPECT Manual***.
For detailed information on using Debug, refer to the Tandem ***Debug Manual***.

Screen 1

DBG screen 1 allows you to place a process under control of Debug or Inspect. To place a process in Debug or Inspect, enter the symbolic process name as configured on Process File (PRO) screen 1 in the PROCESS NAME field and press the **F1** key. When successfully entered, the phrase “Command Processed Successfully” is displayed at the bottom of the screen and the selected process is placed under control of the Inspect or Debug program.

DBG screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
DEBUG
LLLL
YY/MM/DD
HH:MM
01 OF 01

ENTER THE SYMBOLIC NAME OF THE PROCESS TO SEND THE DEBUG COMMAND TO.

PROCESS NAME:

BASE24

FILE DESTINATION:
NEW LOGICAL NETWORK ID:

F1-ENTER
F12-HELP

PROCESS NAME — The symbolic name of the process to be placed under control of Inspect or Debug. This name must match the symbolic name of a process configured in the Process File (PRO).

Field Length:	1–16 alphanumeric characters
Required Field:	Yes
Default Value:	No default value
Data Name:	Not applicable

ACI Worldwide Inc.

Section 8

EMT/Object Relationship File (EORF)

The EMT/Object Relationship File (EORF) defines relationships between all objects and extended memory tables that can be reallocated in the ITS environment using the EMT WARMBOOT command. The file contains one record for each object and extended memory table combination. The records in this file are used to display all the objects that use an extended memory table on EMT Warmboot (EWB) screen 2.

Note: Only extended memory tables that can be reallocated using the EMT WARMBOOT command entered from the EMT Warmboot screen need to be related in the EORF. These extended memory tables are configured on EMT screen 1 with the EMT WARMBOOT ALLOWED field set to the value Y. The EORF is delivered with all extended memory table and object relationships required for kernel objects and ACI-developed applications preconfigured. These default records are described at the end of this section.

After you select the extended memory table to be reallocated on EMT Warmboot screen 1, the EMT Warmboot Server process reads the EORF and displays each object that uses the extended memory table on EMT Warmboot screen 2. You can then select the objects from this list to reallocate the extended memory table.

For each extended memory table and object relationship defined in the EORF, corresponding records must already exist in the Extended Memory Table File (EMT) for the extended memory table, and in the Object File (OBJ) for each of the objects that use the extended memory table.

Records in the EORF must be deleted before you can delete the associated EMT and OBJ records.

The EORF is secured under the Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to EORF records is a combination of the EMT NAME and the OBJECT NAME fields.

Screen 1 Function Keys

The use of one function key on this screen varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The **F5** key is not valid on this screen. To update an EORF record, you must delete the existing record and then add a new record with the updated information.

Screen 1

EORF screen 1 allows you to establish relationships between extended memory tables that can be reallocated using the EMT WARMBOOT command and objects for your ITS environment. EORF screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
EMT/OBJECT RELATIONSHIP
LLLL
YY/MM/DD
HH:MM
01 OF 01

EMT NAME:

(<EMT description>)

OBJECT NAME:

(<object description>)

***** BASE24 *****

NEW PAGE:

FILE DESTINATION:

NEW LOGICAL NETWORK ID:

F12-HELP

EMT NAME — The name of this extended memory table. This name must match a name already assigned to an extended memory table in the Extended Memory Table File (EMT).

Field Length:

1–8 alphanumeric characters

Required Field:

Yes

Default Value:

No default value

Data Name:

EORF.PRIKEY.EMT-NAM

<EMT description> — A description of the extended memory table from the EMT. This description is retrieved from the EMT after a valid extended memory table name is entered in the EMT NAME field.

Field Length:

System-protected

Data Name:

EMT.EMT-DESCR

OBJECT NAME — The name of an object that uses this extended memory table. This name must match a name already assigned to an object in the Object File (OBJ).

Field Length: 1–4 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: EORF.PRIKEY.ALTKEY.OBJ-NAM

<object description> — A description of the object from the OBJ. This description is retrieved from the OBJ after a valid object name is entered in the OBJECT NAME field.

Field Length: System-protected
Data Name: OBJ.OBJ-DESCR

Default Records

The following preconfigured default records for the EORF are provided by ACI. These default records are reserved for use by ACI and its customers and should not be altered for custom development.

EMT NAME	OBJECT NAME
CUSTEMT	CSAC
DESTEMT	DEST
ESEMT	EVT
ESEMT	LOGP
FPFEMT	FC
IMTEMT	IMTT
INTFEMT	IMGR
INTFEMT	IMS
IRTEEMT	RTE
ISECEMT	TSEC
KEYAEMT	ASAC
SCMEMT	CLAS
SSMEMT	XPN

ACI Worldwide Inc.

Section 9

EMT/Process Type Relationship File (EPRF)

The EMT/Process Type Relationship File (EPRF) defines relationships between extended memory tables that can be reallocated using the EMT WARMBOOT command and all process types that access these tables. The records in this file are used to validate the process type for processes to receive an EMT WARMBOOT command. There is one EPRF record for each extended memory table that can be reallocated using the EMT WARMBOOT command and each process type that uses the extended memory table.

Note: Only extended memory tables that can be reallocated using the EMT WARMBOOT command need to be related in the EPRF. These tables are configured on EMT screen 1 with a value of Y in the EMT WARMBOOT ALLOWED field. The EPRF is delivered with all extended memory table and process type relationships required for kernel objects and ACI-developed applications preconfigured. These default records are described at the end of this section.

After you select the objects to receive the EMT WARMBOOT command on EMT Warmboot screen 2, the EMT Warmboot Server process uses the EPRF to validate the process type of processes that contain the selected objects. If a matching record is not found in the EPRF, the associated process does not receive the EMT WARMBOOT command.

For each relationship defined in the EPRF, corresponding records must already exist in the Extended Memory Table File (EMT) and in the Process Type File (PTF). Records in the EPRF must be deleted before you can delete the associated EMT and PTF records.

The EPRF is secured under Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to EPRF records is a combination of the EMT NAME and the PROCESS TYPE fields.

Screen 1 Function Keys

The use of one function key on this screen varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The **F5** key is not valid on this screen. To update an EPRF record, you must delete the existing record and then add a new record with the updated information.

Screen 1

EPRF screen 1 allows you to establish relationships between extended memory tables that can be reallocated using the EMT WARMBOOT command and process types for your ITS environment. EPRF screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
EMT/PROCESS RELATIONSHIP LLLL
YY/MM/DD HH:MM 01 OF 01

EMT NAME:

(<EMT description>)

PROCESS TYPE:

(<process type descrip>)

***** BASE24 *****

NEW PAGE:

FILE DESTINATION:

NEW LOGICAL NETWORK ID:

F12-HELP

EMT NAME — The name of this extended memory table. This name must match a name already assigned to an extended memory table in the EMT.

Field Length:1–8 alphanumeric characters

Required Field:Yes

Default Value:No default value

Data Name:EPRF.PRIKEY.EMT-NAM

<EMT description> — A description of the extended memory table from the EMT. This description is retrieved from the EMT after a valid extended memory table name is entered in the EMT NAME field.

Field Length:System-protected

Data Name:EMT.EMT-DESCR

PROCESS TYPE — The type of process that uses this extended memory table. This name must match a process type name already assigned in the Process Type File (PTF). A process type of all asterisks (*****) indicates that the extended memory table can be used by any process type.

Field Length: 1–8 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: EORF.PRIKEY.ALTKEY.PRO-TYP

<process type descrip> — A description of the process type from the PTF. This description is retrieved from the PTF when a valid process type is entered in the PROCESS TYPE field. When all asterisks are entered in the PROCESS TYPE field, the value WILDCARD is displayed in this field.

Field Length: System-protected
Data Name: PTF.PRO-TYP-DESCR

Default Records

The following preconfigured default records for the EPRF are provided by ACI. These default records are reserved for use by ACI and its customers and should not be altered for custom development.

EMT NAME	PROCESS TYPE
CUSTEMT	IAS
DESTEMT	IEP
ESEMT	*****
FPFEMT	FC
IMTEMT	*****
IMTEMT	IAS
INTFEMT	*****
IRTEEMT	IAS
ISECEMT	IAS
ISECEMT	IMP
KEYAEMT	IAS
SCMEMT	*****
SSMEMT	*****

ACI Worldwide Inc.

Section 10

EMT Warmboot (EWB) Screen

The EMT Warmboot (EWB) screen allows you to select extended memory tables for which the processes that access those extended memory tables are to receive the EMT WARMBOOT command. An EMT WARMBOOT command should be issued any time a change is made to the extended memory table data stored on disk and the extended memory table is rebuilt in memory by the Extended Memory Table Build utility. The rebuilt extended memory table will not be used in processing until the table is reallocated by the processes that access the table.

The EMT WARMBOOT command causes an object to validate the name of the extended memory table, allocate the new extended memory table, deallocate the old extended memory table, and replace the extended memory table segment ID in the frame. If an extended memory table is shared, the object simply reallocates the table when it receives the EMT WARMBOOT command. If the extended memory table is not shared, the object rebuilds the table when it receives the EMT WARMBOOT command. If the allocation of a new extended memory segment fails, then the segment currently in use is not deallocated so that the object can continue processing.

If more complicated processing is required for an extended memory table than that described above (for example, building a secondary frame), then you must use the LCONF WARMBOOT command to reallocate the table. The LCONF WARMBOOT command is entered from the LCONF Warmboot (LWB) screen. The following default extended memory tables provided by ACI use secondary frames and can only be reread using the LCONF WARMBOOT command:

- Institution Definition File (IDF) extended memory table
- Transaction History File (THF) extended memory table
- Transaction Log File (TLF) extended memory table

The context extended memory table is another default extended memory table provided by ACI that can be rebuilt and reread only by using the LCONF WARMBOOT command or by restarting the Integrated Endpoint process. The context extended memory table is internal to the Context Manager object and is not built from a data disk file.

The Interface Security File (ISF) extended memory table is a default extended memory table provided by ACI that can be rebuilt when the Transaction Security object receives either the EMT Warmboot or LCONF WARMBOOT command.

Extended memory tables used by kernel objects are listed by object in the ***BASE24 ITS Kernel Objects Reference Manual***. Extended memory tables used by integrated endpoint objects are listed by object in the ***BASE24 IEP Objects Reference Manual***. Extended memory tables used by core objects are listed in the application-specific transaction processing manual.

The EMT Warmboot Server process reads the following ITS files to process and validate an EMT Warmboot request. These files must be properly configured prior to using the EMT Warmboot function.

- Extended Memory Table File (EMT)
- Extended Memory Table/Object Relationship File (EORF)
- Extended Memory Table/Process Type Relationship File (EPRF)
- Object File (OBJ)
- Process File (PRO)
- Process Type/Object Relationship File (PORF)
- Server Class Type File (SCT)

When a process receives the EMT WARMBOOT command, the Process Control object sends the EMT Warmboot member function to every object that requires an extended memory table to be reallocated. The receiving object verifies that the extended memory table name is correct and that it applies to the object.

The following screens are used to process an EMT WARMBOOT command:

- EWB screen 1 allows you to select an extended memory table to be reallocated.
- EWB screen 2 allows you to select which objects are to receive the EMT WARMBOOT command.

Screen 1 Function Keys

The use of the following keys on EWB screen 1 varies from the standard function keys explained in the *BASE24 CRT Access Manual*. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects an extended memory table to be reallocated and displays EWB screen 2. You must identify the extended memory table to be allocated by placing the value Y in the SELECT field on the same line as the extended memory table prior to pressing this function key.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of extended memory table entries if there are more entries than will fit on one screen.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of extended memory table entries if there are more entries than will fit on one screen.

Screen 1

EWB screen 1 allows you to select an extended memory table to be reallocated. Only those extended memory tables configured in the EMT with the EMT WARMBOOT FLAG field set to the value Y are displayed on this screen. You select an extended memory table by placing the value Y next to the extended memory table to be reread and pressing the **F1** key. Only one extended memory table can be selected at a time. Upon successful selection of an extended memory table, EMT Warmboot screen 2 is displayed, where you can select the individual objects that need to reallocate the extended memory table.

EWB screen 1 is shown below, followed by a description of each field on the screen.

```
BASE24-KNL      EMT WARMBOOT                LLLL      YY/MM/DD  HH:MM  01 OF 02
```

```
      SELECT THE EMT NAME
```

```
      SELECT  EMT NAME
```

```

      N      CUSTEMT
      N      DESTEMT
      N      ESEMT
      N      FPFEMT
      N      IDFEMT
      N      IMTEM
      N      INTFEMT
      N      IRTEEMT
      N      ISECEMT
      N      KEYAEMT
      N      SCMEMT
      N      SSMEMT

```

```
***** BASE24 *****
```

```
      FILE DESTINATION:      NEW LOGICAL NETWORK ID:
```

```
      F1-ENTER      SF7-PREVIOUS PAGE      SF8-NEXT PAGE      F12-HELP
```

SELECT — A flag indicating whether the extended memory table displayed on the same line of the screen is to be reallocated. Only one extended memory table can be selected to be reallocated at a time. When selected, all objects that can reallocate the extended memory table using the EMT WARMBOOT command are displayed on EMT Warmboot screen 2. Valid values are as follows:

Y = Yes, reallocate this extended memory table.

N = No, do not reallocate this extended memory table.

Field Length: 1 alphabetic character

Occurs: Up to 12 times

Required Field: Yes

Default Value: N

Data Name: Not applicable

EMT NAME — The name of an extended memory table that can be reallocated. All extended memory tables configured in the EMT with the EMT WARMBOOT ALLOWED field set to a value of Y are displayed under this field heading.

Field Length: System-protected

Occurs: Up to 12 times

Data Name: Not applicable

Screen 2 Function Keys

The use of the following keys on EWB screen 2 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects the objects to reallocate the specified extended memory table. You must identify the objects to reallocate the extended memory table by placing the value Y in the SELECT fields on the same line as each object prior to pressing this function key. You can select objects on multiple pages prior to pressing the F1 key.
F15	Return to Page 1 — Displays EWB screen 1.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of object entries if there are more entries than will fit on one screen.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of object entries if there are more entries than will fit on one screen.

Screen 2

EWB screen 2 allows you to select objects to receive the EMT WARMBOOT command. After an extended memory table name is selected from EWB screen 1, the system automatically displays all objects that can reallocate that extended memory table using the EMT WARMBOOT command on EWB screen 2. These objects are configured on OBJ screen 1 with the EMT WARMBOOT FLAG field set to a value of Y.

You can then select objects from this list to receive the EMT WARMBOOT command. All processes with a valid process type that contain those objects selected will then receive the EMT WARMBOOT command for the specified extended memory table.

Note: Some objects have the EMT WARMBOOT FLAG field on OBJ screen 1 set to the value N on purpose so that the EMT WARMBOOT command is not supported for those objects. In general, any object that would need to reallocate space for an extended memory table because of entry of the EMT WARMBOOT command, should not support the EMT WARMBOOT command.

Objects are selected for warmboot processing on this screen by placing the value Y next to the name of each object to receive the EMT WARMBOOT command and pressing the **F1** key. You can use the **Shift-F7** and **Shift-F8** keys to view more object selections, if more exist. Once you have selected all the objects which you want to reallocate the specified extended memory table, press the **F1** key to execute the warmboot. All processes that contain the selected objects then automatically receive the EMT WARMBOOT command for the specified extended memory table.

EWB screen 2 is shown below, followed by a description of each field on the screen.

```
BASE24-KNL      EMT WARMBOOT          LLLL      YY/MM/DD  HH:MM  02 OF 02

      THE OBJECTS LISTED BELOW SHARE THE EMT CHOSEN.  SELECT THOSE
      OBJECTS WHICH REQUIRE WARMBOOTING.

              SELECT  OBJECT DESCRIPTION
              N        EVT

***** BASE24 *****
              FILE DESTINATION:      NEW LOGICAL NETWORK ID:
F1-ENTER  F15-RETURN TO PAGE 1  SF7-PREVIOUS PAGE  SF8-NEXT PAGE  F12-HELP
```

SELECT — A flag indicating whether the object displayed on the same line of the screen is to reallocate the specified extended memory table. Valid values are as follows:

Y = Yes, reallocate the extended memory table for this object in all processes.

N = No, do not reallocate the extended memory table for this object in any processes.

Field Length: 1 alphabetic character

Occurs: Up to 12 times

Required Field: Yes

Default Value: N

Data Name: Not applicable

OBJECT DESCRIPTION — The text description of an object to receive the EMT WARMBOOT command for the selected extended memory table. All objects currently configured to accept the EMT WARMBOOT command for the selected extended memory table are displayed.

Field Length:	System-protected
Occurs:	Up to 12 times
Data Name:	Not applicable

ACI Worldwide Inc.

Section 11

Event Suppression File (ESF)

The Event Suppression File (ESF) defines events that are to be suppressed at either the process or network level. The file contains one record for each event that is to be suppressed in some way. Event suppression is used to reduce the number of times a log message is generated, and is needed in an environment where multiple processes report the same error condition or when an error is continually reported by the same process.

The ESF is used by the Event object in an ITS process when performing event suppression at the local process level. The file is used by the Log Permission object in Log Permission Partition processes to perform event suppression at the network level.

Subsystem IDs (SSIDs) used in the ESF must be defined in the company-level DDLs file (for example, \$SYSTEM.SCRIBE.EMCLDDLs) before they can be used in the ESF. This file should be included with your installation of ITS software. SSIDs must be defined in this file for any new objects you develop for your ITS environment.

SSIDs for kernel objects are provided in the ***BASE24 ITS Kernel Configuration Manual***. SSIDs for integrated endpoint objects are provided in the ***BASE24 IEP Configuration Manual***. SSIDs for core objects are provided in the application-specific transaction processing manual.

Event numbers used in the ESF are defined by object in the online documentation files provided in the OK13LOGM subvolume on your system. Each file on this subvolume documents event messages for a specific BASE24 module or object, including the ITS kernel, integrated endpoint, and core objects. The files are organized according to product module ID (PMID). All kernel product module IDs begin with ISKN, all integrated endpoint product module IDs begin with ISIE, and all core product module IDs begin with ISCO.

Detailed instructions on configuring event suppression are provided in the ***BASE24 ITS Kernel Configuration Manual***.

Warning: Indiscriminate use of event suppression may adversely affect your ability to monitor system processing by limiting your ability to detect significant processing events if and when they should occur. Therefore, any attempts at event suppression should not be undertaken until after you have had significant experience running your ITS software in a production environment and you are familiar with the types of event messages generated.

The ESF is secured under the Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to ESF records is a combination of the SSID and EVENT NUMBER fields.

Screen 1

ESF screen 1 allows you to define an event to be suppressed, including the length of time the event is to be suppressed or the number of events to skip when suppressing the event.

ESF screen 1 is shown below, followed by a description of each field on the screen.

```

BASE24-KNL  EVENT SUPPRESSION FILE  LLLL          YY/MM/DD  HH:MM  01 OF 01
SSID:        (<subsystem description>)  EVENT NUMBER:

                LOG STATE: 0 (LPPCONTROL)

                TIME UNIT: S (SECONDS)

                TIME INTERVAL:      0

NUMBER OF EVENTS TO SKIP:      0

***** BASE24 *****
NEW PAGE:      FILE DESTINATION:      NEW LOGICAL NETWORK ID:
                F12-HELP

```

SSID — The subsystem ID (SSID) associated with an application or application module for which an event is to be suppressed. The subsystem ID (SSID) is assigned to an application or application module in the company-level DDLs file (for example, \$SYSTEM.SCRIBE.EMCLDDLs). Any time you add a new object, an SSID for the object must be added to the EMCLDDLs file. Valid values range from 0 to 65535.

Note: The SSID cannot be the SSID for the Log Permission object.

Field Length:	1–5 alphanumeric characters
Required Field:	Yes
Default Value:	No default value
Data Name:	ESF.PRIKEY.SSID

<subsystem description> — The subsystem description associated with the SSID from the EMCLDDLS file. The text of this field will vary, depending upon whether or not the SYSGEN utility has been run for the Event Management Service (EMS) on the Tandem system. If the SYSGEN utility has been run for EMS, this field will contain a text description of the SSID (for example, ISKANS). If the SYSGEN utility has not been run for EMS, this field will contain a numeric representation of the SSID as discussed in the following sentences. Numeric representations are provided in the format of *ACI.ssid.subsystem-name*, where *ssid* is the SSID from the SSID field or its textual representation, and *subsystem-name* is a number ranging from 0–149 for ACI modules and from 150–299 for ACI RPQ modules.

Example: (ACI.302.0) is a numeric SSID representation
 (ACI.ISKNANS.0) is a text SSID representation

Field Length: System-protected

Data Name: EMCLDDLS.SSID.Z-NUMBER

EVENT NUMBER — The number generated by the SSID for the event that is to be suppressed. If an event is not defined in the ESF, it will not be suppressed. The key to the ESF is the subsystem identifier (SSID) of the event generator and the event number. Valid values range from 0 to 65535.

Field Length: 1–5 numeric characters

Required Field: Yes

Default Value: No default value

Data Name: ESF.PRIKEY.EVT-NUM

LOG STATE — A code specifying the conditions under which the event is logged or not logged (suppressed). The Event object determines whether an event should be suppressed each time it encounters an SSID and event defined in the ESF with a LOG STATE field value of 1 through 3. For events with a LOG STATE field value of 0, the Log Permission object in the Log Permission process determines whether the event is suppressed. Valid values are as follows:

- 0 = Log the event if the time or count threshold has been reached by the Log Permission object in the Log Permission process (network-level suppression).
- 1 = Log the event if the time or count threshold has been reached by the Event object in the local process (process-level suppression).
- 2 = Do not log the event.
- 3 = Log the event.

Field Length: 1 numeric character
Required Field: Yes
Default Value: 0
Data Name: ESF.ALTKEY.LOG-ST

TIME UNIT — The units (seconds or minutes) used for the time interval specified in the TIME INTERVAL field. Valid values are as follows:

S = Seconds
M = Minutes

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: S
Data Name: ESF.DELTA-TIME-MS

TIME INTERVAL — The time interval, in seconds or minutes, for which this event will be suppressed. If the specified log message is logged within this interval, it will be suppressed. A time interval of 0 indicates that suppression by time interval is not used. If this field and the NUMBER OF EVENTS TO SKIP field are both set to nonzero values, whichever of the two occurs first takes precedence over the other. This field will accept values up to 9999 seconds or minutes. Valid values range from 0 to 9999.

Field Length: 1–4 numeric characters
Required Field: Yes, if the LOG STATE field is set to a value of 0 or 1
Default Value: 0
Data Name: ESF.DELTA-TIME-MS

NUMBER OF EVENTS TO SKIP — The number of events to suppress between successive occurrences of this event. For example, if this field is set to 100, 100 attempts to log the event will be suppressed before the Event object or Log Permission object again grants permission to log the event. The default value of 0 indicates that suppression by number of events is not used. If this field and the TIME INTERVAL field are both set to nonzero values, whichever of the two occurs first takes precedence over the other. Valid values range from 0 to 65535.

Field Length: 1–5 numeric characters
Required Field: Yes, if the LOG STATE field is set to a value of 0 or 1
Default Value: 0
Data Name: ESF.SKIP-CNT-MAX

ACI Worldwide Inc.

Section 12

Extended Memory Table File (EMT)

The Extended Memory Table File (EMT) defines all extended memory tables used in the ITS environment that can be built by the EMT Build utility or reallocated using the EMT WARMBOOT command. The EMT contains one record for each extended memory table in the ITS environment that meets these requirements.

The EMT is delivered with all extended memory tables required for kernel objects, integrated endpoint objects, and ACI-developed applications preconfigured. These default records are described at the end of this section.

All extended memory tables are maintained in the extended memory of a Tandem processor in the current system. Extended memory tables typically contain frequently accessed information or information critical to the performance of the Tandem system. Most extended memory tables are built from disk files used as swap files in processor memory. Only shared extended memory tables can be built by the EMT Build utility.

Some default extended memory tables provided by ACI, such as the context extended memory table for the Context Manager object and the Interface Security File (ISF) extended memory table for the Transaction Security object, are not built by the Extended Memory Table Build utility. The context extended memory table is not built from a disk file and is built and reallocated only when the Context Manager object initializes (that is, when it receives the LCONF WARMBOOT command). Therefore, it is not configured in the EMT at all. The ISF extended memory table is specific to an interface and, therefore, is not shared. This extended memory table is built only when the Transaction Security object receives the EMT WARMBOOT or LCONF WARMBOOT command.

The EMT WARMBOOT command causes an object to validate the name of the extended memory table, allocate the new extended memory table, deallocate the old extended memory table, and replace the extended memory table segment ID in the frame. If more complicated processing is required for an extended memory table (for example, building a secondary frame), then you must use the LCONF WARMBOOT command to reallocate the table.

The following default extended memory tables provided by ACI use secondary frames and can only be reread using the LCONF WARMBOOT command.

- Institution Definition File (IDF) extended memory table
- Transaction History File (THF) extended memory table
- Transaction Log File (TLF) extended memory table

If an extended memory table such as the Card Prefix File (CPF) extended memory table provided by ACI is used by a procedural process, it cannot be reallocated by the EMT WARMBOOT or LCONF WARMBOOT commands. In this case, the extended memory table is reallocated only when the procedural process that uses it is stopped and restarted or warmbooted, using the appropriate network environment (that is, Pathway or XPNET) facilities.

Records for the EMT must exist in this file before related records can be added to the following files:

- EMT/Process Type Relationship File (EPRF)
- EMT/Object Relationship File (EORF)

A record for an extended memory table must also exist in the EMT before an extended memory table can be built using the Extended Memory Table Build utility or reallocated using the EMT WARMBOOT command.

Associated records in the EPRF and EORF must be deleted before you can delete an EMT record.

The EMT is secured under Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to EMT records is the EMT NAME field.

Screen 1

EMT screen 1 allows you to define the extended memory tables used in your ITS environment. EMT screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL

EMT FILE

LLLL

YY/MM/DD

HH:MM

01 OF 01

EMT NAME:

EMT DESCRIPTION:

EMT WARMBOOT ALLOWED:

SUPPORTED BY EMT BUILD:

***** BASE24 *****

NEW PAGE:

FILE DESTINATION:

NEW LOGICAL NETWORK ID:

F12-HELP

EMT NAME — The name for this extended memory table.

- Field Length:1–8 alphanumeric characters
- Required Field:Yes
- Default Value:No default value
- Data Name:EMT.PRIKEY.EMT-NAM

EMT DESCRIPTION — A textual description of this extended memory table. This field is used for informational purposes only and is displayed on the screens used to maintain the following files:

- EMT/Object Relationship File (EORF)
- EMT/Process Type Relationship File (EPRF)

Field Length: 1–20 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: EMT.EMT-DESCR

EMT WARMBOOT ALLOWED — A flag indicating whether this extended memory table can be reallocated using the EMT WARMBOOT command. Valid values are as follows:

- Y = Yes, this extended memory table can be reallocated using the EMT WARMBOOT command.
N = No, this extended memory table cannot be reallocated using the EMT WARMBOOT command.

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: Y
Data Name: EMT.EMT-WB-ALWD

SUPPORTED BY EMT BUILD — A flag indicating whether this extended memory table can be built using the Extended Memory Table Build utility. Valid values are as follows:

- Y = Yes, this extended memory table can be built using the Extended Memory Table Build utility.
N = No, this extended memory table cannot be built using the Extended Memory Table Build utility.

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: Y
Data Name: EMT.EMTB-SUPPORT

Default Records

The following preconfigured default records for the EMT are provided by ACI. These default records are reserved for use by ACI and its customers and should not be altered for custom development.

EMT NAME	EMT DESCRIPTION	EMT WARMBOOT ALLOWED	SUPPORTED BY EMT BUILD
CPFEMT	CARD PREFIX FILE	N	Y
CUSTEMT	CUSTOMER PROFILE	Y	Y
DESTEMT	DESTINATION	Y	Y
ESEMT	EVENT SUPPRESSION	Y	Y
FPFEMT	FRAUD PARAMETER FILE	Y	Y
IDFEMT	INSTITUTION DEF	N	Y
IMTEMT	INT MSG TRANSLATION	Y	Y
INTFEMT	INTERFACE	Y	Y
IRTEMT	INSTITUTION ROUTING	Y	Y
ISECEMT	INTERFACE SECURITY	Y	N
KEYAEMT	AUTHORIZATION KEY	Y	Y
SCMEMT	SOURCE CLASS MAP	Y	Y
SSMEMT	SOURCE SYMBOLIC MAP	Y	Y
THFEMT	TRANSACTION HISTORY	N	Y
TLFEMT	TRANSACTION LOG FILE	N	Y

ACI Worldwide Inc.

Section 13

LCONF Warmboot (LWB) Screen

The LCONF Warmboot (LWB) screen allows you to select objects to receive the LCONF WARMBOOT command. This command causes objects to perform initialization processing. During initialization, objects read all applicable Logical Network Configuration File (LCONF) assigns and params and allocate extended memory tables. Using this screen, you can read all the LCONF assigns and params and allocate all extended memory tables for a single object, a few objects, or all objects in the ITS environment.

The LCONF WARMBOOT command can be issued for the following reasons:

- Multiple LCONF assigns, LCONF params, or extended memory tables for a particular object have changed.
- You do not know exactly which specific LCONF assigns, LCONF params, or extended memory tables have changed.
- You need to reinitialize an object.

An LCONF WARMBOOT command may also be required for any assigns, params, or extended memory tables that cannot be reread or reallocated individually using the ASSIGN WARMBOOT, PARAM WARMBOOT, or EMT WARMBOOT commands, respectively.

Only objects that accept the LCONF WARMBOOT command and have the LCONF WARMBOOT FLAG field set to a value of Y on Object File (OBJ) screen 1 can be used on the LCONF Warmboot screen. If an object is configured on OBJ screen 1 with the LCONF WARMBOOT FLAG field set to a value of N, it will not appear as an object selection on the LCONF Warmboot screen.

When a process receives the LCONF WARMBOOT command, the Process Control object sends the LCONF Warmboot member function to every object selected to be reinitialized. This member function contains the name of the LCONF and the symbolic name of the process to which the command applies.

If an error occurs while an object is performing LCONF warmboot processing, the object will continue processing with its old LCONF and extended memory table configuration.

The LCONF Warmboot Server process reads the following files to determine which processes contain those objects selected and to validate those processes. Records must be properly defined in these files before the LCONF Warmboot screen can be used.

- Process/Object Relationship File (PORF)
- Process File (PRO)
- Server Class Type/Process Relationship File (SPRF)
- Server Class Type File (SCT)

LCONF assigns, LCONF params, and extended memory tables used by kernel objects are listed by object in the ***BASE24 ITS Kernel Objects Reference Manual***. Assigns, params, and extended memory tables used by integrated endpoint objects are listed by object in the ***BASE24 IEP Objects Reference Manual***. Assigns, params, and extended memory tables used by core objects are listed in the application-specific transaction processing manual. LCONF assign and param descriptions are provided in the ***BASE24 Logical Network Configuration File Manual***.

The LWB screen is secured under Tandem group level security so that only authorized network operators can access the LCONF Warmboot function.

Screen 1 Function Keys

The use of the following keys on LWB screen 1 varies from the standard function keys explained in the *BASE24 CRT Access Manual*. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects the objects to reinitialize. You must identify the objects to reinitialize by placing the value Y in the SELECT fields on the same line as each object prior to pressing this function key. You can select objects on multiple pages prior to pressing the F1 key.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of object entries if there are more entries than will fit on one screen.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of object entries if there are more entries than will fit on one screen.

Screen 1

When you first enter LWB screen 1, the LCONF Warmboot Server process reads the Object File (OBJ) and determines which objects have the LCONF WARMBOOT FLAG field set to the value Y. A list of all applicable objects is then displayed. You select objects for warmboot processing by placing the value Y next to the name of each object to be reinitialized and pressing the **F1** key. More than one object can be selected at a time. You can use the **Shift-F7** and **Shift-F8** keys to view more object selections, if there are more entries than will fit on one screen. Once you have selected all the objects you want to be reinitialized, press the **F1** key to execute the warmboot.

When the LCONF WARMBOOT command is successfully sent, the message “Command Processed Successfully” is displayed at the bottom of the screen.

LWB screen 1 is shown below, followed by a description of each field on the screen.

```

BASE24-KNL   LCONF WARMBOOT           LLLL           YY/MM/DD  HH:MM  01 OF 01

THE OBJECTS LISTED BELOW SUPPORT LCONF WARMBOOTS.  SELECT THOSE
OBJECTS WHICH REQUIRE WARMBOOTING.

                SELECT      OBJECT DESCRIPTION
                N           ACCOUNT ACCESS
                N           AUTH SECURITY ACCESS
                N           CLASS
                N           CUSTOMER SERVICE
                N           CUSTOMER SVC INTF
                N           CUSTOMER
                N           DESTINATION
                N           END OF PERIOD
                N           EVENT
                N           FINANCIAL INST ACC
                N           FRAUD CONTROL

***** BASE24 *****
                FILE DESTINATION:      NEW LOGICAL NETWORK ID:
F1-ENTER      SF7-PREVIOUS PAGE      SF8-NEXT PAGE      F12-HELP

```

SELECT — A flag indicating whether the object described on the same line of the screen is to be reinitialized. Valid values are as follows:

Y = Yes, reinitialize this object in all processes.

N = No, do not reinitialize this object in any process.

Field Length: 1 alphabetic character

Occurs: Up to 12 times

Required Field: Yes

Default Value: N

Data Name: Not applicable

OBJECT DESCRIPTION — The description of an object to be reinitialized. All objects currently configured on Object File (OBJ) screen 1 with the LCONF WARMBOOT FLAG field set to the value Y are displayed in alphabetical order.

Field Length: System-protected

Occurs: Up to 12 times

Data Name: Not applicable

ACI Worldwide Inc.

Section 14

Log Permission Process Partition File (LPF)

The Log Permission Process Partition File (LPF) contains a record for each range of subsystem IDs (SSIDs) used in the system to be handled by a Log Permission process for network-level suppression. The file maps ranges of SSIDs to specific Log Permission processes (LPPs) responsible for arbitration when determining whether an event is to be logged at the network level. This allows log permission requests to be spread across multiple Log Permission processes, effectively distributing the workload of network-level event suppression processing.

The range of SSIDs assigned to a Log Permission process for arbitration consists of the first SSID assigned to the Log Permission process up to the first SSID established for another Log Permission process. For example, if the first SSID for P1A^LPP1 is 100, and the next record in the LPF establishes a first SSID of 200 for P1A^LPP2, the range of SSIDs from 100 through 199 will be handled by the P1A^LPP1 process.

Subsystem IDs (SSIDs) used in the LPF must be defined to the company-level DDLs file (for example, \$SYSTEM.SCRIBE.EMCLDDLs) before they can be used in the LPF. This file should be included with your installation of ITS software. SSIDs must be defined in this file for any new objects you develop for your ITS environment.

Detailed instructions on configuring event suppression are provided in the ***BASE24 ITS Kernel Configuration Manual***.

Warning: Indiscriminate use of event suppression may adversely affect your ability to monitor system processing by limiting your ability to detect significant processing events if and when they should occur. Therefore, any attempts at event suppression should not be undertaken until after you have had significant experience running your ITS software in a production environment and you are familiar with the types of event messages generated.

The LPF is secured under the Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to LPF records is the FIRST SSID field.

Screen 1

LPF screen 1 allows you to establish the first SSID in the range of SSIDs for which a particular Log Permission process will arbitrate network-level event suppression. LPF screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
LPP PARTITION FILE
LLLL
YY/MM/DD
HH:MM
01 OF 01

FIRST SSID:
(<subsystem description>
)

LPP ASSIGN NAME:

***** BASE24 *****
NEW PAGE:
FILE DESTINATION:
NEW LOGICAL NETWORK ID:
F12-HELP

FIRST SSID — The first subsystem identification number (SSID) in the range of SSIDs handled by this Log Permission process. If all SSIDs are to be handled by the same Log Permission process, this file must contain only one record with this field set to a value of 0. If SSIDs are to be handled by separate Log Permission processes, a record must be entered for the first SSID in each range of SSIDs to be handled by the same Log Permission process. All SSIDs from the first SSID in the range up to the first SSID named in the next record are considered to be in the same range. Valid values are 0 through 65535.

SSIDs are defined in the EMCLDDLS file for your system (for example, \$SYSTEM.SCRIBE.EMCLDDLS). Any time a new object is added to the ITS environment, an SSID for the object must be added to the EMCLDDLS file.

Field Length:	1–5 numeric characters
Required Field:	Yes
Default Value:	No default value
Data Name:	LPF.PRIKEY.SSID-FIRST

<subsystem description> — The subsystem description associated with the SSID in the company-level DDLs file (that is, the EMCLDDLS file). The text of this field will vary, depending upon whether or not the SYSGEN utility has been run for the Event Management Service (EMS) on the Tandem system. If the SYSGEN utility has been run for EMS, this field will contain a text description of the SSID (for example, ISKANS for the Application Network Services object). If the SYSGEN utility has not been run for EMS, this field will contain a numeric representation of the SSID as discussed in the following sentences. Numeric representations are provided in the format of *ACI.ssid.subsystem-name*, where *ssid* is the SSID from the SSID field or its textual representation, and *subsystem-name* is a number ranging from 0–149 for ACI modules and from 150–299 for ACI RPQ modules.

Example: (ACI.302.0) is a numeric SSID representation
 (ACI.ISKNANS.0) is a textual SSID representation

Field Length: System-protected

Data Name: EMCLDDLS.SSID.Z-NUMBER

LPP ASSIGN NAME — The symbolic name of the Log Permission process that is to arbitrate network-level event suppression for events generated from this range of SSIDs. The symbolic name entered here is used to key into the LCONF to access the actual PPD name of the Log Permission process. The LPP assign name must begin with the LPP- characters, followed by the symbolic process name.

Example: LPP-P1A^LPP1

Field Length: 1–32 alphanumeric characters

Required Field: Yes

Default Value: No default value

Data Name: LPF.LPP-ASGN-NAM

Section 15

Object File (OBJ)

The Object File (OBJ) defines all objects used in the ITS environment and their process control capabilities. All kernel, IEP, and core objects used in your ITS environment must be defined in the OBJ. Each record contains an object description, object ID, various processing flags that determine how processing is performed for the object, and the trace levels supported for the TRACE command. Objects must be defined in the OBJ before related records can be added to the following files:

- Source Class Map File (SCM)
- Assign/Object Relationship File (AORF)
- Process/Object Relationship File (PORF)
- EMT/Object Relationship File (EORF)

All associated records for an object must be deleted from the above files before the OBJ record for an object can be deleted.

The OBJ is delivered with all object records required for kernel objects and ACI-developed applications preconfigured. These default records are described at the end of this section.

Each OBJ record contains several flags defining the network operations capabilities for the object. For example, the WARMBOOT LCONF FLAG field indicates whether the object supports the LCONF WARMBOOT command. The STATUS COMMAND FLAG field indicates whether the object supports the STATUS command.

Note: No flags are provided in the OBJ to indicate support of the ASSIGN WARMBOOT or the EMT WARMBOOT commands. An object supports the ASSIGN WARMBOOT command only if it is coded to do so and an Assign/Object Relationship File (AORF) record exists for the assign and object. An object supports the EMT WARMBOOT command only if it is coded to do so and an EMT/Object Relationship File (EORF) record exists for the EMT and object.

The OBJ is used extensively by the Process Control subsystem to determine how processing is to be performed for specific objects when performing various network operation tasks.

The OBJ is secured under the Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to OBJ records is the OBJECT NAME field.

The following screens are used to access records in the OBJ:

- OBJ screen 1 describes the object and defines the process control commands the object supports.
- OBJ screen 2 defines the trace levels supported by the object for object tracing.

Screen 1

OBJ screen 1 allows you to define the characteristics of each object used in your ITS environment. OBJ screen 1 is shown below, followed by a description of each field on the screen.

```

BASE24-KNL  OBJECT FILE          LLLL      YY/MM/DD  HH:MM  01 OF 02

                OBJECT NAME:

                        OBJECT ID:
                OBJECT DESCRIPTION:
                WARMBOOT LCONF FLAG: N
                WARMBOOT PARAM FLAG: N
                STATUS COMMAND FLAG: N
                GENERAL COMMAND FLAG: N
                        OBJECT TYPE:

***** BASE24 *****
NEW PAGE:          FILE DESTINATION:          NEW LOGICAL NETWORK ID:
                        F12-HELP

```

OBJECT NAME — The name of the object being defined.

Field Length: 1–4 alphanumeric characters
 Required Field: Yes
 Default Value: No default value
 Data Name: OBJ.PRIKEY.OBJ-NAM

OBJECT ID — An identification number for the object being defined, which is unique with the following exceptions:

- The ANS and Space object share the same object ID as they are physically the same object. Therefore, the Space object is not configured.
- A terminal EPCH object shares the same object ID as its corresponding terminal transaction object in the ITS process.
- All components of an interchange interface (the EPCH, the interface object, and the interface master object) share the same object ID.

- All Transport Layer objects (TLOs) share the same object ID, since only one TLO can reside in a particular ITS process.

The object ID is used internally by the ITS software to identify objects. The OBJECT ID field is used as an alternate key into the OBJ. Valid values are 1 through 32767.

Ranges of object IDs have been established by ACI for products as follows. Object IDs in the range of 1–19,999 are reserved exclusively for objects developed for ACI products. Channel-developed objects should use the range of 20,000–29,999. Customer-developed objects should use the range of 30,000–31,999 if the objects are a result of an RPQ, or the range of 32,000–32,765 for custom products. Object IDs in ACI-provided default records should not be changed.

Product	Object ID Range
Ghost object [*]	0
Kernel (TAL)	1–249
Kernel (C++)	250–499
Integrated Endpoint (TAL)	500–749
Integrated Endpoint (C++)	750–999
Core (TAL)	1,000–1,499
BASE24-telebanking (TAL)	1,500–1,999
Core (C++)	2,000–6,999
Reserved (for ACI products)	7,000–19,999
Channel [†]	20,000–29,999
Customer RPQ	30,000–31,999
Reserved (for custom products)	32,000–32,765
Simulator Object [‡]	32,766
Yang Object [§]	32,767

- * The ghost object does not represent an actual object. The object ID for the ghost object is used internally by the Application Network Services object to vary slot IDs when it registers object IDs in the Object Control Table (OCB). The Application Network Services object provides the main procedure for an object-oriented process in the ITS environment. Among its functions, it oversees initialization, memory acquisition, and inter-object communication processing.
- † A channel is an organization within ACI that is responsible for customizing, delivering, and supporting ACI products for customers in specific geographic regions.
- ‡ The Simulator object allows developers to test an object's interactions with objects that are not yet completed or are not available for testing.
- § The Yang object is an educational tool that exhibits fundamental object architecture; however, it is not intended to be used as an object template for creating new objects.

Field Length: 1–5 numeric characters
 Required Field: Yes
 Default Value: No default value
 Data Name: OBJ.ALTKEY.OBJ-ID

OBJECT DESCRIPTION — A description of this object. This value is for informational purposes only; it is displayed whenever a valid object name is entered on any ITS kernel file maintenance screen.

Field Length: 1–20 alphanumeric characters
 Required Field: Yes
 Default Value: No default value
 Data Name: OBJ.OBJ-DESCR

WARMBOOT LCONF FLAG — A flag indicating whether this object can receive the LCONF WARMBOOT command. If so, the object reinitializes when the LCONF WARMBOOT command is issued to the object from the LCONF Warmboot screen (LWB). Reinitialization causes the object to read its applicable assigns and params from the LCONF as well as reallocate its extended memory tables. Valid values are as follows:

Y = Yes, this object can receive the LCONF WARMBOOT command.
 N = No, this object cannot receive the LCONF WARMBOOT command.

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: N
Data Name: OBJ.LCONF-WB-FLAG

WARMBOOT PARAM FLAG — A flag indicating whether this object can receive the PARAM WARMBOOT command. If so, the object rereads the LCONF for its params that can be warmbooted when the PARAM WARMBOOT command is issued to the object from the Param Warmboot screen (PWB). Valid values are as follows:

Y = Yes, this object can receive the PARAM WARMBOOT command.
N = No, this object cannot receive the PARAM WARMBOOT command.

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: N
Data Name: OBJ.PARAM-WB-FLAG

STATUS COMMAND FLAG — A flag indicating whether this object can receive STATUS commands. If so, the object will output its current status in Event Management Service (EMS) events when the STATUS command is issued to the object from the Status screen (STAT). Valid values are as follows:

Y = Yes, this object can receive the STATUS command.
N = No, this object cannot receive the STATUS command.

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: N
Data Name: OBJ.STAT-CMD-FLAG

GENERAL COMMAND FLAG — A flag indicating whether this object can receive any of the general commands issued from the Server Control screen (SCS). See the SCS description elsewhere in this manual, for descriptions of all general commands available from this screen. If at least one general command can be received by an object, then this field must be set to the value Y. Valid values are as follows:

Y = Yes, this object can receive at least one general SCS command.
N = No, this object cannot receive any general SCS commands.

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: N
Data Name: OBJ.GENRL-CMD-FLG

OBJECT TYPE — Codes that define the traits (for example, interface or terminal) or type of this object. Depending upon the object's function, an object can have no traits or multiple traits defined. When calling an object, the ITS software must make sure it is of the correct object type. Kernel objects typically do not have an object type. Object types are used mainly for application-specific (that is, core) objects.

The following traits have been defined by ACI. Additional traits can be defined if needed. Any character in the range of A–Z and 0–9 is available if it has not already been defined below. Values A–F are reserved for kernel object types. Values G–Z and 0–9 are reserved for core object types. Traits currently defined include the following:

A = Secure data object
0 = Acquirer
1 = ATM
2 = Authorization
3 = Configurable ISO
4 = Interface
5 = Issuer
6 = POS
7 = Telebanking
8 = Terminal

Field Length: 1 alphanumeric character
Occurs: Up to 10 times
Required Field: No
Default Value: No default value
Data Name: OBJ.OBJ-TYP

Screen 2

OBJ screen 2 identifies the trace levels for the object. OBJ screen 2 is shown below, followed by descriptions of each field appearing on the screen.

```

BASE24-KNL   OBJECT FILE           LLLL           YY/MM/DD  HH:MM  02 OF 02

                OBJECT NAME:

                SET THE TRACE LEVELS WHICH ARE SUPPORTED BY THE OBJECT.

1.  Y    9.  N   17.  N   25.  N   33.  N   41.  N   49.  N   57.  N
2.  Y   10.  N   18.  N   26.  N   34.  N   42.  N   50.  N   58.  N
3.  N   11.  N   19.  N   27.  N   35.  N   43.  N   51.  N   59.  N
4.  N   12.  N   20.  N   28.  N   36.  N   44.  N   52.  N   60.  N
5.  N   13.  N   21.  N   29.  N   37.  N   45.  N   53.  N   61.  N
6.  N   14.  N   22.  N   30.  N   38.  N   46.  N   54.  N   62.  N
7.  N   15.  N   23.  N   31.  N   39.  N   47.  N   55.  N   63.  N
8.  N   16.  N   24.  N   32.  N   40.  N   48.  N   56.  N   64.  N

***** BASE24 *****
NEW PAGE:           FILE DESTINATION:           NEW LOGICAL NETWORK ID:
                      F12-HELP

```

TRACE LEVEL — A series of flags indicating the trace levels supported when the TRACE command is sent to this object. All objects implement tracing for trace levels 1–4.

Trace data is added to the trace files or trace terminal through the Trace Add member function in the Object to Trace lexicon. The data appropriate for each global trace level is defined below with the trace level.

Trace levels 1 through 16 are reserved for globally supported trace levels. All objects support these trace levels as defined in the following table. Any exceptions to this rule are noted. Valid values for each trace level field are as follows:

Y = Yes, this object supports this trace level.

N = No, this object does not support this trace level.

Trace Level	Action
1	Trace all procedure calls. For every procedure the object enters, a trace record is written containing the procedure name. Subprocedures are not traced. This trace level is supported by all objects.
2	Trace all object interactions. The Application Network Services object traces request and response buffers during response processing prior to responding to a client. Objects which have security data in their requests or responses indicate to the Application Network Services object during registration that they do not participate in trace level 2. These objects set the Trace Exclusion flag in the Register member function in the Object to ANS object lexicon to a value of True. The Application Network Services object traces request or response data only if both the client object and the actor object participate in level 2 tracing. If either object does not participate in level 2 tracing, request and response data is not traced. This trace level is supported, as a service, by the Application Network Services object.
3	Trace all file input and output. Each trace includes the file name and input or output function (that is, read, write, control, setmode, open, or close). In addition, record buffers are traced for read and write operations. Control and setmode trace records include parameters used in the operation. File input and output used during the processing of warmboot requests is also traced. Only objects that perform file input and output operations support this trace level.
4	Trace all interprocess input and output. Each trace includes the process name and input or output function (that is, read, writeread, control, setmode, open, or close). In addition, message buffers are traced for read, write, and writeread operations. Control and setmode trace records include parameters used in the operation. Only objects that send or receive interprocess messages support this trace level.

Trace Level	Action
5	Trace Internal Transaction Data (ITD). Tracing of ITD is provided as a service of the Transaction object. Objects which have trace level 5 enabled set the TRC-ITD flag in Object to Transaction lexicon member functions to a value of True, causing the Transaction object to trace the ITD.
6	Trace External Message In. This trace level causes the field contents for an inbound message to be traced. This trace level is used by all issuer and acquirer objects, but is not a required function. For ISO-based messages, the Message ISO object performs the tracing for the client acquirer or issuer object. For non-ISO based messages, the actual acquirer or issuer object is responsible for tracing the data as it is moved into the ITD.
7	Trace External Message Out. This trace level causes the field contents for an outbound message to be traced. This trace level is used by all issuer and acquirer objects, but is not a required function. For ISO-based messages, the Message ISO object performs the tracing for the client acquirer or issuer object. For non-ISO based messages, the actual acquirer or issuer object is responsible for tracing the data as it is moved out of the ITD.
8–16	Reserved for future use of globally supported trace levels.
17–32	Reserved for object-specific trace levels. These traces must be defined at the object level.
33–64	Reserved for future use.

Field Length: 1 alphabetic character
Occurs: 64 times
Required Field: Yes
Default Value: Y for trace levels 1 and 2
N for all other trace levels
Data Name: OBJ.TRC-LVL

Default Records

The following preconfigured default records for the OBJ are provided by ACI. These default records are reserved for use by ACI and its customers and should not be altered for custom development.

OBJECT NAME	OBJECT ID	OBJECT DESCRIPTION	OBJECT TYPE	WARMBOOT LCONF FLAG	WARMBOOT PARAM FLAG	STATUS COMMAND FLAG	GENERAL COMMAND FLAG	TRACE LEVELS SUPPORTED
ACAC	1002	ACCOUNT ACCESS		Y	N	N	N	1–3
ACCT	1001	ACCOUNT		N	N	N	N	1, 2, 5
ANS	1	APPL NETWK SERVICES		N	N	Y	N	1,2
ASAC	1003	AUTH SECURITY ACCESS		Y	N	N	N	1–3
CLAS	2	CLASS		Y	N	N	N	1–3
CSAC	1025	CUSTOMER ACCESS		Y	Y	N	Y	1–3
CSI	1004	CUSTOMER SVC INTF	0, 4, 7	Y	Y	N	N	1–3, 5
CTXM	501	CONTEXT MANAGER		N	N	Y	N	1–3
CUST	1005	CUSTOMER		Y	Y	N	N	1–3, 5
DENA	1006	DENIAL AUTH	2, 7	N	N	N	N	1–2, 5
DEST	502	DESTINATION		Y	N	N	N	1–3
EOPP	1007	END OF PERIOD		Y	Y	N	Y	1–3
EPCS	503	EPCH SERVICES		N	N	N	N	1–2
EVT	3	EVENT		Y	Y	N	N	1–4

OBJECT NAME	OBJECT ID	OBJECT DESCRIPTION	OBJECT TYPE	WARMBOOT LCONF FLAG	WARMBOOT PARAM FLAG	STATUS COMMAND FLAG	GENERAL COMMAND FLAG	TRACE LEVELS SUPPORTED
FC	1024	FRAUD CONTROL		Y	N	N	N	1-3
FI	1008	FINANCIAL INST		N	N	N	N	1-2, 5
FIAC	1009	FINANCIAL INST ACC		Y	N	N	N	1-3
GENE	504	GENERIC EPCH		N	N	N	N	1-2
HIST	1010	TRANSACTION HISTORY		Y	N	N	N	1-3, 5
IMGR	1011	INTERFACE MANAGER		Y	Y	N	N	1-4
IMS	1012	INTF MASTER SERVICES		Y	N	N	Y	1-3
IMTE	1013	INTERN MSG XLATE EPCH	7	N	N	N	N	1-2
IMTT	1013	INTERN MSG XLATE INTF	4, 5, 7	Y	N	N	N	1-3, 5
LOGP	4	LOG PERMISSION		Y	N	N	N	1-3
MC	1015	MULTIPLE CURRENCY		N	N	N	N	1-2
MISO	505	MESSAGE ISO		N	N	N	N	1-2
PCA	1017	POSITIVE CUST AUTH	2, 7	N	N	N	N	1-2, 5
PCAA	1018	PSTV CUST ACCT AUTH	2, 7	N	N	N	N	1-2, 5
PCBA	1019	PSTV CUST BAL AUTH	2, 7	N	N	N	N	1-2, 5
PCTL	5	PROCESS CONTROL		Y	N	N	N	1-3
RTE	1020	TRANSACTION ROUTER		Y	N	N	N	1-3, 5

OBJECT NAME	OBJECT ID	OBJECT DESCRIPTION	OBJECT TYPE	WARMBOOT LCONF FLAG	WARMBOOT PARAM FLAG	STATUS COMMAND FLAG	GENERAL COMMAND FLAG	TRACE LEVELS SUPPORTED
SPC	1	SPACE		N	N	Y	N	
TRC	8	TRACE		Y	N	N	N	
TSEC	1021	TRANSACTION SECURITY	A	Y	Y	Y	N	1–3, 5
TXN	1022	TRANSACTION		Y	Y	N	N	1–3
VRU	1023	VRU	0, 3, 4, 7	Y	Y	N	N	1–3, 5–7
VRUE	1023	VRU EPCH	7	N	N	N	N	1–3, 6–7
VRUM	1023	VRU MASTER	7	N	N	N	N	1–2, 6–7
VRUT	1023	VRU INTERFACE	0, 3, 4, 7	Y	Y	N	N	1–3, 5–7
XPN	7	XPNET		Y	Y	N	N	1–4

ACI Worldwide Inc.

Section 16

Param Warmboot (PWB) Screen

The Param Warmboot (PWB) screen allows you to select objects to receive the PARAM WARMBOOT command. This command causes objects to reread certain params from the Logical Network Configuration File (LCONF). Using this screen, you can reread the LCONF params for a single object, a few objects, or all objects in the ITS environment.

A param can be reread using this screen any time the value of the param is changed and the object that uses the param supports the PARAM WARMBOOT command. Only objects that have the PARAM WARMBOOT FLAG field set to a value of Y on Object File (OBJ) screen 1 are displayed on the Param Warmboot screen. If an object is configured on OBJ screen 1 with the PARAM WARMBOOT FLAG field set to a value of N, it will not appear as an object selection on PWB screen 1.

The intent of the PARAM WARMBOOT command is to simply swap the old value for the param with the new value, without performing any reinitialization processing. Params that cannot be reread using the PARAM WARMBOOT command must be reread using the LCONF WARMBOOT command.

LCONF params used by kernel objects are listed by object in the ***BASE24 ITS Kernel Objects Reference Manual***. Params used by integrated endpoint objects are listed by object in the ***BASE24 IEP Objects Reference Manual***. Params used by core objects are listed in the application-specific transaction processing manual. Individual LCONF param descriptions are provided in the ***BASE24 Logical Network Configuration File Manual***.

When a process receives the PARAM WARMBOOT command, the Process Control object sends the Param Warmboot member function to every object selected in the process. This member function contains the internal format of the LCONF name and the symbolic name of the process for which the command is intended.

Records must be properly defined in the following files before the Param Warmboot screen can be used:

- Object File (OBJ)
- Process File (PRO)
- Process/Object Relationship File (PORF)
- Server Class Type/Process Relationship File (SPRF)
- Server Class Type File (SCT)

The PWB is secured under the Tandem group level security so that only authorized network operators can access the Param Warmboot function.

Screen 1 Function Keys

The use of the following function keys on this screen varies from the standard function keys explained in the *BASE24 CRT Access Manual*. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects the objects to receive the PARAM WARMBOOT command. You must identify the objects to receive the command by placing the value Y in the SELECT fields on the same line as each object prior to pressing this function key. You can select objects on multiple pages prior to pressing the F1 key.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of object entries if there are more entries than will fit on one screen.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of object entries if there are more entries than will fit on one screen.

Screen 1

PWB screen 1 allows you to select the objects to receive the PARAM WARMBOOT command. Objects are selected for warmboot processing on PWB screen 1 by placing the value Y next to the name of each object to be warmbooted and pressing the **F1** key. You can use the **Shift-F7** and **Shift-F8** keys to view more object selections, if there are more entries than will fit on one screen. More than one object can be selected at a time. Once you have selected all the objects you want to be warmbooted, press the **F1** key to execute the warmboot. All ITS processes containing the selected objects then receive the PARAM WARMBOOT command.

When the PARAM WARMBOOT command is successfully sent, the message “Command Processed Successfully” is displayed at the bottom of the screen.

PWB screen 1 is shown below, followed by a description of each field on the screen.

```

BASE24-KNL   PARAM WARMBOOT           LLLL           YY/MM/DD  HH:MM  01 OF 01

      THE OBJECTS LISTED BELOW SUPPORT PARAM WARMBOOTS.  SELECT THOSE
      OBJECTS WHICH REQUIRE WARMBOOTING.

                SELECT      OBJECT DESCRIPTION
                N           CUSTOMER ACCESS
                N           CUSTOMER SRV INTF
                N           CUSTOMER
                N           END OF PERIOD
                N           EVENT
                N           INTERFACE MANAGER
                N           TRANSACTION SECURITY
                N           TRANSACTION
                N           VRU
                N           VRU INTERFACE
                N           XPNET

***** BASE24 *****
                FILE DESTINATION:      NEW LOGICAL NETWORK ID:
F1-ENTER      SF7-PREVIOUS PAGE      SF8-NEXT PAGE      F12-HELP

```

SELECT — A flag indicating whether the object displayed on the same line of the screen is to reread its applicable LCONF params. Valid values are as follows:

Y = Yes, reread the applicable LCONF params for this object in all processes.

N = No, do not reread the applicable LCONF params for this object in any process.

Field Length: 1 alphabetic character

Occurs: Up to 12 times

Required Field: Yes

Default Value: N

Data Name: Not applicable

OBJECT DESCRIPTION — A description of an object to be reread from the OBJ. All objects currently configured on Object File (OBJ) screen 1 with the PARAM WARMBOOT FLAG field set to the value Y are displayed.

Field Length: System-protected

Occurs: Up to 12 times

Data Name: Not applicable

ACI Worldwide Inc.

Section 17

Process File (PRO)

The Process File (PRO) defines all processes used in the ITS environment, including both ACI- and customer-defined processes, that can receive process control commands. The PRO contains one record for each process used in the ITS environment. Each entry in the Process File defines the Tandem Process Pair Directory (PPD) name assigned to the process and the type of process (from the Process Type File).

If an object-oriented ITS process runs under control of the XPNET process, it must be defined in this file. No preconfigured data is provided for the PRO. You must enter all applicable records for your system in this file during or after installation.

Note: A PRO record must be configured for any procedural processes (for example, the Enscribe Refresh process) that use the Process Control subsystem to receive command messages.

TRACE, DEBUG, and STATUS commands can only be executed against processes defined in the PRO.

An entry must exist for the process type in the Process Type File (PTF) before the process can be entered here. In addition, processes must be defined in the PRO before corresponding entries can be defined in each of the following files:

- Process/Object Relationship File (PORF)
- Server Class/Process Relationship File (SPRF)

Records in the PRO cannot be deleted until the associated records in the PORF and SPRF have been deleted.

The PRO is used extensively during process control processing to retrieve the process name associated with a process.

The PRO is secured under the Tandem group level security so that only authorized network operators can access, or start programs that access, this file. The key to PRO records is the PROCESS NAME field.

Screen 1

PRO screen 1 allows you to define each process in your ITS environment. PRO screen 1 is shown below, followed by a description of each field on the screen.

```

BASE24-KNL   PROCESS FILE           LLLL           YY/MM/DD  HH:MM  01 OF 01

                PROCESS NAME:

                    PPD NAME:
                PROCESS TYPE:           <process type description>
                    QUEUE FLAG: N

***** BASE24 *****
NEW PAGE:           FILE DESTINATION:           NEW LOGICAL NETWORK ID:
                    F12-HELP

```

PROCESS NAME — The symbolic name for the process being defined. This field should match the symbolic name assigned to the process in the XPNET system.

Field Length: 1–16 alphanumeric characters
 Required Field: Yes
 Default Value: No default value
 Data Name: PRO.PRIKEY.PRO-NAM

PPD NAME — The Tandem Process Pair Directory (PPD) name assigned to this process in the XPNET system.

The value in this field must be a dollar sign (\$) followed by up to 5 unique alphanumeric characters (for example, \$T1I1), unless a remote PPD name is entered. If a remote PPD name is entered, then the PPD name must be prefaced by a system name followed by a dollar sign (\$) and up to 4 unique alphanumeric characters (for example, \ACI2.\$T1I1).

Field Length: 2–13 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: PRO.PPD-NAM

PROCESS TYPE — The type of this process. The process type provides a means of categorizing similar processes for process control purposes. The process type entered in this field must match a process type already defined in the Process Type File (PTF).

Field Length: 1–8 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: PRO.ALTKEY.PRO-TYP

<process type description> — A description of this process type. This description is retrieved from the Process Type File after a valid process type is entered in the PROCESS TYPE field.

Field Length: System-protected
Data Name: PTF.PRO-TYP-DESCR

QUEUE FLAG — This field is no longer used, but must contain a valid value. Valid values are as follows:

Y = Yes, this process functions as a Queue process for incoming messages.
N = No, this process does not function as a Queue process for incoming messages.

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: N
Data Name: PRO.QUEUE-FLG

ACI Worldwide Inc.

Section 18

Process/Object Relationship File (PORF)

The Process/Object Relationship File (PORF) defines relationships between processes and objects. An entry must exist in this file for each occurrence of an object in an ITS process. This file is accessed during ASSIGN WARMBOOT, PARAM WARMBOOT, LCONF WARMBOOT, EMT WARMBOOT, and STATUS command processing to automatically determine the processes associated with the objects to which the WARMBOOT or STATUS command is directed. Corresponding records must exist in the Object File (OBJ) and in the Process File (PRO) before they can be related here. There is one PORF record for each process and object relationship in the ITS environment.

No preconfigured data is provided for the PORF. You must enter all applicable records for your system in this file during or after installation.

To set up the PORF, you can use the following methods to determine which objects are required for each process.

- Check the MAKE files used to create the process. MAKE files indicate all of the objects that comprise the process. Refer to the ***BASE24 ITS Kernel Configuration Manual*** for a list of the MAKE files used for each type of object-oriented process in the ITS environment.
- Use the Tandem Binder product to determine the objects that make up an executable process (known as an object). All executable object files end with a file name of LO and can be found in the object subvolume for your system (for example, \$DATA.PRO1OBJ). For detailed information on the Binder product, refer to the Tandem ***Binder Manual***.

Records in the PORF must be deleted before you can delete the associated OBJ and PRO records.

The PORF is secured under the Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to PORF records is a combination of the PROCESS NAME and OBJECT NAME fields.

Screen 1 Function Keys

The use of one function key on this screen varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The **F5** key is not valid on this screen. To update a PORF record, you must delete the existing record and then add a new record with the updated information.

Screen 1

PORF screen 1 allows you establish relationships between processes and objects for your ITS environment. PORF screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL

PROCESS/OBJECT RELATION

LLLL

YY/MM/DD

HH:MM

01 OF 01

PROCESS NAME:

OBJECT NAME:

<object description>

***** BASE24 *****

NEW PAGE:

FILE DESTINATION:

NEW LOGICAL NETWORK ID:

F12-HELP

PROCESS NAME — The name of a process. The name entered here must match a process name already defined in the Process File (PRO). This field should contain the symbolic process name as defined to the XPNET system.

- Field Length:

1–16 alphanumeric characters
- Required Field:

Yes
- Default Value:

No default value
- Data Name:

PORF.PRIKEY.PRO-NAM

OBJECT NAME — The name of an object being related to this process. The name entered here must match an object name already defined in the Object File (OBJ).

Field Length: 1–4 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: PORF.PRIKEY.ALTKEY.OBJ-NAM

<object description> — A description of the object. This description is retrieved from the OBJ when a valid object name is entered in the OBJECT NAME field.

Field Length: System-protected
Data Name: OBJ.OBJ-DESCR

Section 19

Process Type File (PTF)

The Process Type File (PTF) defines all process types used in the ITS environment, including ACI-defined and customer-defined process types, to the process control subsystem. Process types must be defined in this file before they can be used anywhere else.

The process type is typically a category for a particular type of process and is used by the Process Control subsystem to determine which processes should receive particular commands. For example, all ITS Integrated Authorization Server processes could be defined with a process type of IAS while all Integrated Endpoint processes could be defined with a process type of IEP. In this example, commands entered against an IAS process type will be executed only against Integrated Authorization Server processes while commands entered with an IEP process type will be executed only against Integrated Endpoint processes. When configuring the process type in other files, you can use a process type of all asterisks (*****) to represent all process types configured in the PTF. Each asterisk specifies a wild card character.

The PTF is delivered with all process types required for kernel and ACI-developed applications preconfigured. These default records are described at the end of this section.

Note: The default PTF records do not include procedural processes. A PTF record must be configured for any procedural processes (for example, the Enscribe Refresh process) that use the Process Control subsystem to receive command messages.

The PTF is used by the Process Control subsystem to validate processes and to ensure that operator- and process-initiated commands are written to the appropriate Integrated Server Control Files (ISCFs).

All object-oriented ITS processes must have a process type defined in this file. In addition, the process type must exist in this file before related records in the following files can be configured:

- Assign/Process Type Relationship File (APRF)
- EMT/Process Type Relationship File (EPRF)
- Process File (PRO)

Records in the PTF cannot be deleted until after all associated records have been deleted from the APRF, EPRF, and PRO.

The PTF is secured under Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to PTF records is the PROCESS TYPE field.

Screen 1

PTF screen 1 allows you to define each of the process types used in your ITS environment. PTF screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
PROCESS TYPE FILE
LLLL
YY/MM/DD
HH:MM
01 OF 01

PROCESS TYPE:

PROCESS TYPE DESCRIPTION:

BASE24

NEW PAGE:
FILE DESTINATION:
NEW LOGICAL NETWORK ID:

F12-HELP

PROCESS TYPE — An acronym defining a particular category of processes to the Process Control subsystem.

In files relating the process type to other system entities (that is, the Assign/Process Type Relationship File and the EMT/Process Type Relationship File), eight wildcard characters can be used (*****) to represent all process types configured in the PTF. Therefore, the wildcard character should not be used in this field.

- Example: IAS
- Field Length: 1–8 alphanumeric characters
- Required Field: Yes
- Default Value: No default value
- Data Name: PTF.PRIKEY.PRO-TYP

PROCESS TYPE DESCRIPTION — A textual description of this process type. This field is used for informational purposes only and is displayed on the screens used to maintain the following files:

- Assign/Process Type Relationship File (APRF)
- EMT/Process Type Relationship File (EPRF)

When a process type of eight asterisks is entered in either of the above files, a description of WILDCARD is displayed on the screens used to maintain them. Therefore, this description should not be entered in this field.

Example:	Integrated Auth Srv
Field Length:	1–20 alphanumeric characters
Required Field:	Yes
Default Value:	No default value
Data Name:	PTF.PRO-TYP-DESCR

Default Records

The following preconfigured default records for the PTF are provided by ACI. These default records are reserved for use by ACI and its customers and should not be altered for custom development.

PROCESS TYPE	PROCESS TYPE DESCRIPTION
EOPP	END OF PERIOD
FC	FRAUD CONTROL
IAS	INTEGRATED AUTH SRV
IEP	INTEGRATED ENDPOINT
IMP	INTERFACE MASTER
LPP	LOG PERMISSION

ACI Worldwide Inc.

Section 20

Server Class Type/Process Relationship File (SPRF)

The Server Class Type/Process Relationship File (SPRF) defines all the relationships between server class types and processes in the ITS environment. Only processes that are identical in nature and object composition should be related to the same server class. Each process can be related to only one server class. All processes of the same server class type look at the same Integrated Server Control File (ISCF) for commands entered by the Process Control subsystem.

The SPRF is used extensively by the Process Control subsystem to determine the server class type associated with a particular process. The server class type is then used to retrieve the associated Integrated Server Control File (ISCF) to which a particular operator- or process-initiated command is to be written.

No preconfigured data is provided for the SPRF. You must enter all applicable records for your system in this file after installation.

Note: SPRF records must be configured for any procedural processes (for example, the Enscribe Refresh process) that use the Process Control subsystem to receive command messages.

All server class types used in this file must first be defined in the Server Class Type File (SCT). All process names used in this file must first be defined in the Process File (PRO).

Records in the SPRF must be deleted before you can delete the associated SCT and PRO records.

The SPRF is secured under Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to SPRF records is a combination of the SERVER CLASS TYPE and the PROCESS NAME fields.

Screen 1 Function Keys

The use of one function key on the SPRF screen varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The **F5** key is not valid on this screen. To update an SPRF record, you must delete the existing record and then add a new record with the updated information.

Screen 1

SPRF screen 1 allows you to establish the relationships between server class types and processes for your ITS environment. SPRF screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
SRV CLASS/PRO RELATION
LLLL
YY/MM/DD
HH:MM
01 OF 01

SERVER CLASS TYPE:
(<server class type description>)

PROCESS NAME:

BASE24

NEW PAGE:
FILE DESTINATION:
NEW LOGICAL NETWORK ID:

F12-HELP

SERVER CLASS TYPE — An acronym defining a particular class of identical servers. The server class type entered here must already be defined in the Server Class Type File (SCT).

Field Length: 1–16 alphanumeric characters
 Required Field: Yes
 Default Value: No default value
 Data Name: SPRF.PRIKEY.SRV-CLAS-TYP

<server class type description> — A description of the server class from the Server Class Type File (SCT). This description is retrieved from the SCT after a valid server class type is entered in the SERVER CLASS TYPE field.

Field Length: System-protected
 Data Name: SCT.SRV-CLAS-TYP-DESCR

PROCESS NAME — The symbolic name of a process being related to this server class type. This name must match the symbolic name of a process already defined in the Process File (PRO).

Field Length: 1–16 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: SPRF.PRIKEY.ALTKEY.PRO-NAM

Section 21

Server Class Type File (SCT)

The Server Class Type File (SCT) defines all server class types used in the ITS environment to the Process Control subsystem, including both ACI-defined and any customer-defined server class types. Each server class type accesses a different Integrated Server Control File (ISCF). Each record contains the server class type, a description of the server class type, and the LCONF assign for the control file associated with the server class type.

Server classes are used to define a set of identical servers that perform the same processing and have the same composition of objects. Thus, when a process control command is sent to a server class to warmboot a particular object, all processes belonging to that server class will process the same command and remain synchronized. Other server classes will not receive the command if the object to be warmbooted does not belong to that server class. Therefore, the other server classes are not interrupted by any unnecessary messages. The SCT is used extensively by the Process Control subsystem to retrieve the name of an ISCF associated with a particular server class. All network operator- and process-initiated commands are delivered to processes using ISCFs.

If a process control command such as the TRACE command is sent to a particular object, all processes belonging to the server class will receive the message. The only process that will act on the message, however, is the process with the PPD name specified when the command was issued. Certain commands impact a single process within a server class, while others impact all processes belonging to the server class.

No preconfigured data is provided for the SCT. You must enter all applicable records for your system in this file during or after installation.

Note: An SCT record also must be configured on behalf of any procedural processes (for example, the Enscribe Refresh process) that use the Process Control subsystem to receive command messages.

All ITS server classes must have a server class type defined in this file. In addition, the server class type must exist in this file before server classes can be related to processes in the Server Class Type/Process Relationship File (SPRF).

All associated entries in the SPRF must be deleted before an SCT record can be deleted.

The SCT is secured under the Tandem group level security so that only authorized network operators can access, or start programs that access, this file.

The key to SCT records is the SERVER CLASS TYPE field.

Screen 1

SCT screen 1 allows you to define each server class type used in your ITS environment, including the name of the Integrated Server Control File (ISCF) used by the server type. SCT screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
SERVER CLASS TYPE FILE
LLLL
YY/MM/DD
HH:MM
01 OF 01

SERVER CLASS TYPE:

SERVER CLASS TYPE DESCRIPTION:

CONTROL FILE ASSIGN NAME:

CONTROL FILE NUMBER: 0

***** BASE24 *****

NEW PAGE:

FILE DESTINATION:

NEW LOGICAL NETWORK ID:

F12-HELP

SERVER CLASS TYPE — An acronym defining a particular class of identical ITS servers.

- Example: IAS
- Field Length: 1–16 alphanumeric characters
- Required Field: Yes
- Default Value: No default value
- Data Name: SCT.PRIKEY.SRV-CLAS-TYP

SERVER CLASS TYPE DESCRIPTION — A description of this server class. This field is for informational purposes only; it is not used during processing.

Example: Integrated Auth Svr
Field Length: 1–20 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: SCT.SRV-CLAS-TYP-DESCR

CONTROL FILE ASSIGN NAME — The name of the LCONF assign specifying the fully qualified physical location of the Integrated Server Control File (ISCF) for this server class. The control file is the mechanism used to deliver process control commands. A unique ISCF exists for each server class in the system. Only members of a particular server class access this ISCF. All ISCF assign names must always begin with ISCF- followed by a user-defined name for the server class the file services. An example of an ISCF assign for Integrated Authorization Server processes could be ISCF-IAS.

Field Length: 1–32 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: SCT.CNTL-FILE-ASGN-NAM

CONTROL FILE NUMBER — A single-digit sequence number for the ISCF. ISCFs are numbered from 0 to 9 and are automatically rolled over to the next number when the current ISCF is full. After the number 9 is used, the file number rolls back to number 0. Each time the control file is rolled to a new number, the current data in the next ISCF is purged. This field is for display only.

Field Length: System-protected
Data Name: SCT.CNTL-FILE-NUM

Section 22

Server Control (SCS) Screen

The Server Control (SCS) screen allows you to send general server control commands such as logons or key exchanges to processes in the ITS environment. These commands cannot be entered through the XPNET network control facilities.

Using SCS screen 1, you enter the name of the process to receive the command, the name of the command function, and any data associated with the function. Upon successful entry of this data, SCS screen 2 is displayed for you to select which objects are to receive the command in the specified process.

Only objects that have the GENERAL COMMAND FLAG field on Object File (OBJ) screen 1 set to a value of Y are displayed as an object selection on SCS screen 2. If an object is configured on OBJ screen 1 with the GENERAL COMMAND FLAG field set to a value of N, it will not appear as an object selection on SCS screen 2.

The Process Control subsystem uses the following files to automate the processing of a command entered on the SCS screen. Records must be properly defined in these files before the Server Control screen can be used.

- Object File (OBJ)
- Process/Object Relationship File (PORF)
- Process File (PRO)
- Server Class Type/Process Relationship File (SPRF)
- Server Class Type File (SCT)

When a command function is successfully entered, a message is displayed indicating whether the command was successfully sent (for example, "Command Processed Successfully").

All commands entered from the SCS are delivered to the targeted process by writing the command information to the appropriate Integrated Server Control Files (ISCFs).

The following screens are used to process server control commands:

- SCS screen 1 allows you to enter the name of the process to receive the command as well as the command control function name and any associated data.
- SCS screen 2 allows you to select the objects to receive the server control command in the specified process.

Screen 1

SCS screen 1 allows you to send server control commands to processes in your ITS environment. SCS screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
SERVER CONTROL
LLLL
YY/MM/DD
HH:MM
01 OF 02

PROCESS NAME:

FUNCTION NAME:

FUNCTION DATA:

***** BASE24 *****

FILE DESTINATION:
NEW LOGICAL NETWORK ID:

F1-ENTER
F12-HELP

PROCESS NAME — The name of the ITS process on which the command specified in the FUNCTION NAME field is to be performed.

- Field Length: 1–16 alphanumeric characters
- Required Field: Yes
- Default Value: No default value
- Data Name: Not applicable

FUNCTION NAME — The name of the command function to be executed on this process. All server commands are object-specific. For kernel-specific command functions, refer to the *BASE24 ITS Kernel Objects Reference Manual*. For IEP-specific command functions, refer to the *BASE24 IEP Objects Reference Manual*. For core-specific command functions, refer to the product-specific transaction processing manual.

When you enter a function and press the **F1** key, a message is displayed indicating the success or failure of the command (for example, “Command Completed Successfully”).

Field Length: 1–16 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: Not applicable

FUNCTION DATA — Contains any required data for the specified command function. The data necessary for each command function varies. Refer to the manuals mentioned in the FUNCTION NAME field description for this data.

Field Length: 1–50 alphanumeric characters
Required Field: No
Default Value: No default value
Data Name: Not applicable

Screen 2 Function Keys

The use of the following function keys on SCS screen 2 varies from the standard function keys explained in the *BASE24 CRT Access Manual*. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects the objects in the specified ITS process to receive the command entered on SCS screen 1. You must identify the objects to receive the command by placing the value Y in the SELECT fields on the same line as each object prior to pressing this function key. You can select objects on multiple pages prior to pressing the F1 key.
F15	Return to Page 1 — Displays SCS screen 1.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of object entries if there are more entries than will fit on one screen.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of object entries if there are more entries than will fit on one screen.

Screen 2

SCS screen 2 allows you to select objects to receive the server control command function and data entered on SCS screen 1. After a process name, server control function, and function data are entered on screen 1, the Server Control Server process reads the Process/Object Relationship File (PORF) and determines all the objects for the process entered on SCS screen 1. Then, all of these objects that have the GENERAL COMMAND FLAG field set to the value Y on Object File (OBJ) screen 1 are displayed on this screen.

You select objects to receive the specified control function by placing the value Y next to the name of each object to receive the command and pressing the **F1** key. If more than one page of objects exist, you can use the **Shift-F7** and **Shift-F8** keys to view more object selections. Once you have selected all the objects on which the specified function is to be executed, press the **F1** key to execute the function.

SCS screen 2 is shown below, followed by a description of each field on the screen.

```
BASE24-KNL  SERVER CONTROL          LLLL  YY/MM/DD  HH:MM  02 OF 02
```

```
SELECT THE OBJECTS WHICH NEED TO RECEIVE THE COMMAND.
```

```
SELECT      OBJECT DESCRIPTION
```

```
    N      CUSTOMER ACCESS
```

```
***** BASE24 *****
```

```
FILE DESTINATION:      NEW LOGICAL NETWORK ID:
```

```
F1-ENTER  F15-RETURN TO PAGE 1  SF7-PREVIOUS PAGE  SF8-NEXT PAGE  F12-HELP
```


SELECT — Indicates whether the object displayed on the same line of the screen is to receive the specified command function and data. Valid values are as follows:

Y = Yes, send the specified command to this object.

N = No, do not send the specified command to this object.

Field Length: 1 alphabetic character

Occurs: Up to 12 times

Required Field: Yes

Default Value: N

Data Name: Not applicable

OBJECT DESCRIPTION — The text description of an object on which the specified command function is to be executed. All objects currently configured to the selected process that have the GENERAL COMMAND FLAG field set to the value Y on Object File (OBJ) screen 1 are displayed.

Field Length: System-protected

Occurs: Up to 12 times

Data Name: Not applicable

ACI Worldwide Inc.

Section 23

Source Class Map File (SCM)

The Source Class Map File (SCM) is used to relate source class values defined for processes and stations in the XPNET system with objects in the ITS environment. For systems running under XPNET 3.0, the source class is derived from the SOURCECLASS attribute. The source class is a user-defined grouping of processes or stations from which object-oriented ITS processes can receive messages. A source class provides a mechanism for ensuring that messages from a particular source are processed by a particular message handler object in the ITS environment. This file is used by the Class object to map a source class to its associated message handler object. Source class map processing is performed by the Class object in every object-oriented ITS process when a message is received, unless the object ID of the message handler object is provided in the message itself.

The Source Class Map File (SCM) contains one record for each source class from which object-oriented ITS processes can receive messages. Each record contains the object ID of the object which is to receive messages from the given source class.

In general, source classes should be mapped to ITS objects as follows:

- Interface stations are mapped to the interface-specific Endpoint Class Handler (EPCH) object.
- Terminal stations are mapped to the terminal-specific EPCH object.
- Object-oriented processes (for example, Integrated Endpoint, Integrated Authorization Server, and Interface Master processes) are mapped to the XPNET Interface object.
- Procedural processes are mapped to the Internal Message Translate Interface (IMTT) object.

No preconfigured data is provided for the SCM. You must enter all applicable records for your system in this file after installation.

Source class values must first be defined to the XPNET system. Object names must first be defined in the Object File (OBJ).

The SCM is secured under Tandem group level security so that only authorized network operators can access, or start programs which access, this file.

The key to SCM records is the SOURCE CLASS field.

Screen 1

SCM screen 1 allows you to map each source class used in your ITS environment to a message handler object. SCM screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
SOURCE CLASS MAP FILE
LLLL
YY/MM/DD
HH:MM
01 OF 01

SOURCE CLASS:

OBJECT NAME: (<object description>)

***** BASE24 *****

NEW PAGE:
FILE DESTINATION:
NEW LOGICAL NETWORK ID:

F12-HELP

SOURCE CLASS — The source class assigned to a message source (that is, process or station) as defined to the XPNET system. The source class is associated with a station or process from which messages originate. A source class derived from the SOURCECLASS attribute can be up to four characters in length.

Field Length:
1–16 alphanumeric characters

Required Field:
Yes

Default Value:
No default value

Data Name:
SCM.PRIKEY.SRC-CLAS

OBJECT NAME — The name of an ITS object that receives messages from this source class. A record must exist for this object in the OBJ before the object name can be entered here.

Note: This field is redefined with the object ID of the object by the Extended Memory Table Build utility when it builds the Source Class Map File extended memory table. The first two bytes of the object name are overlaid with the object ID associated with the object. This allows the object ID, which is an integer field, to be used internally for faster processing.

Field Length: 1–4 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: SCM.ALTKEY.OBJ-NAM

<object description> — A description of this object from the OBJ. This description is retrieved from the OBJ when a valid object name is entered in the OBJECT NAME field.

Field Length: System-protected
Data Name: OBJ.OBJ-DESCR

Section 24

Status (STAT) Screen

The Status (STAT) screen allows you to send the STATUS command to objects within an ITS process. The information generated by the STATUS command can vary with each object that receives the command. For example, in response to a STATUS command, the Application Network Services object, a kernel object, displays the general status of the object, including the number of bytes available for the process in processor memory. All response data to the STATUS command is generated using Event Management Service (EMS) events.

Only objects that have the STATUS COMMAND FLAG field set to a value of Y on Object File (OBJ) screen 1 can be selected using the Status screen. If an object is configured on OBJ screen 1 with the STATUS COMMAND FLAG field set to a value of N, it will not appear as an object selection on STAT screen 2.

When a process receives the STATUS command, the Process Control object sends the Status member function to every selected object in the process.

Proper use of the STAT screens assumes that the appropriate records are already configured in the following files. These files are accessed by the Process Control subsystem when processing a STATUS command.

- Process File (PRO)
- Process/Object Relationship File (PORF)
- Object File (OBJ)
- Server Class Type/Process Relationship File (SPRF)
- Server Class Type File (SCT)

When the STATUS command is successfully sent, the message “Command Processed Successfully” is displayed on the screen.

The STAT screen is secured under the Tandem group level security. Only authorized network operators can access the object status function.

The following screens are used to perform STATUS commands:

- STAT screen 1 allows you to enter the name of the process to receive the STATUS command.
- STAT screen 2 allows you to specify the objects in the targeted process to receive the STATUS command.

Screen 1

STAT screen 1 allows you to specify an ITS process to which the STATUS command is sent. After entering a symbolic process name in the PROCESS NAME field, press the **F1** key to validate the name. If the process name is valid, the Process Control system automatically determines which objects in the process can receive the STATUS command and displays these objects on STAT screen 2.

STAT screen 1 is shown below, followed by a description of each field on the screen.

BASE24-KNL
STATUS COMMAND
LLLL
YY/MM/DD
HH:MM
01 OF 02

ENTER THE SYMBOLIC NAME OF THE PROCESS TO SEND THE STATUS COMMAND TO.

PROCESS NAME:

BASE24

FILE DESTINATION:
NEW LOGICAL NETWORK ID:

F1-ENTER
F12-HELP

PROCESS NAME — The symbolic name of the process to which the STATUS command is to be sent. The name entered in this field must match a symbolic process name configured in the Process File (PRO).

- Example: P1A^IAS01
- Field Length: 1–16 alphanumeric characters
- Required Field: Yes
- Default Value: No default value
- Data Name: Not applicable

Screen 2 Function Keys

The use of the following function keys on STAT screen 2 varies from the standard function keys explained in the *BASE24 CRT Access Manual*. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects the objects within the ITS process to receive the STATUS command. You must identify the objects to receive the STATUS command by placing the value Y in the SELECT fields on the same line as each object prior to pressing this function key. You can select objects on multiple pages prior to pressing the F1 key.
F15	Return to Page 1 — Displays STAT screen 1.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of object entries if there are more than 12 objects in the specified process.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of object entries if there are more than 12 objects in the specified process.

Screen 2

STAT screen 2 allows you to select the objects within the specified process to which the STATUS command is to be sent. Note that some objects do not support the STATUS command. The Process Control subsystem determines which objects are contained in the specified process by reading the Process/Object Relationship File (PORF). It determines which objects support the STATUS command by reading the Object File (OBJ).

After the process name is selected on screen 1, the system displays all objects that support the STATUS command for the selected process on screen 2. Only objects with the STATUS COMMAND FLAG field set to the value Y on Object File (OBJ) screen 1 are displayed. Objects are selected by placing the value Y next to the name of each object and pressing the **F1** key. You can use the **Shift-F7** and **Shift-F8** keys to view more object selections, if more than 12 entries exist. Once you have selected all the objects for which you want the status, press the **F1** key to send the STATUS command to the selected objects. Upon pressing the **F1** key, a message indicating the success or failure of the command is displayed back to the screen (for example, "Command Processed Successfully").

STAT screen 2 is shown below, followed by a description of each field on the screen.

BASE24-KNL
STATUS COMMAND
LLLL
YY/MM/DD
HH:MM
02 OF 02

SELECT THE OBJECTS WHICH NEED TO RECEIVE THE COMMAND.

SELECT	OBJECT DESCRIPTION
N	APPL NETWK SERVICES

***** BASE24 *****

FILE DESTINATION:
NEW LOGICAL NETWORK ID:

F1-ENTER
F15-RETURN TO PAGE 1
SF7-PREVIOUS PAGE
SF8-NEXT PAGE
F12-HELP

SELECT — A flag indicating whether the object displayed on the same line of the screen is to receive the STATUS command. Valid values are as follows:

Y = Yes, send the STATUS command to this object.

N = No, do not send the STATUS command to this object.

Field Length: 1 alphabetic character

Occurs: Up to 12 times

Required Field: Yes

Default Value: N

Data Name: Not applicable

OBJECT DESCRIPTION — The text description of an object residing in the specified process that supports the STATUS command. All objects currently configured to the specified process with the STATUS COMMAND FLAG field set to the value Y on OBJ screen 1 will be displayed.

Field Length: System-protected

Occurs: Up to 12 times

Data Name: Not applicable

Status Command Response Data

The information returned from the STATUS command varies from object to object and is reported using one or more EMS event messages. Event message documentation for the Integrated Transaction Services product is contained in files located in the OK13LOGM subvolume. Each file contains event messages for a specific BASE24 module or object, including the ITS kernel, integrated endpoint, and core modules and objects. The files are organized according to product module ID (PMID). Kernel PMIDs begin with ISKN, integrated endpoint PMIDs begin with ISIE, and core PMIDs begin with ISCO.

ACI Worldwide Inc.

Section 25

Trace Control (TRC) Screen

The Trace Control (TRC) screen allows you to start, stop, and inquire about traces executed on objects within an ITS process. Depending upon the level of trace supported, traces are used to record the actions taken by particular objects in the course of transaction processing. The trace levels supported vary from object to object. Tracing can be performed on any number of objects and levels supported in the targeted process. Traces are used as a tool in debugging and to verify that transaction processing is proceeding as anticipated. The response data generated by the TRACE command varies from object to object and can be displayed on a terminal or written to a trace file. Interpretation of trace results can be complicated and should only be performed by personnel familiar with the internal processing performed by ITS objects.

Note: A trace cannot be performed on the Trace (TRC) object itself.

Warning: Use of the Trace Control screen may cause some degradation to system performance. Therefore, the trace function should only be used by qualified personnel when needed to pinpoint problems or to verify proper transaction processing performed in the ITS environment. The trace function should normally be used only in test environments. The trace function should not be used in a production environment, unless explicitly requested by ACI for problem resolution.

Proper use of the TRC screen assumes that the appropriate records are already configured in the following files. These files are accessed by the Process Control subsystem when processing TRACE commands.

- Process File (PRO)
- Process/Object Relationship File (PORF)
- Object File (OBJ)
- Server Class Type/Process Relationship File (SPRF)
- Server Class Type File (SCT)

The TRC screens are secured under Tandem group level security so that only authorized network operators can access the object trace function.

The following screens are used to process traces:

- TRC screen 1 allows you to start a trace, stop a trace, or list the objects currently undergoing a trace.
- TRC screen 2 allows you to select the symbolic name of a process to be traced. From this screen, you can also specify trace file parameters.
- TRC screen 3 allows you to select the individual objects to be traced in the targeted process.
- TRC screen 4 allows you to stop a trace currently in progress.
- TRC screen 5 allows you to issue a TRACE LIST command to obtain the status of a trace currently in progress.
- TRC screen 6 allows you to send the TRACE SECURE ITD command to selected objects. This command is used to trace security-related information (for example, PIN validation information).

Trace Levels

All of the possible trace levels for an ITS object are shown in the following table. Trace levels 1 and 2 are supported by all objects. Objects may support additional trace levels as configured for the object in the Object File (OBJ). To determine which levels are supported for an object, see OBJ screen 2 for the particular object. For detailed descriptions of trace levels, see the Object File (OBJ) description elsewhere in this manual.

Trace Level	Description
1	Trace all procedural calls.
2	Trace all object interactions.
3	Trace all disk file and table input and output.
4	Trace all interprocess input and output.
5	Trace Internal Transaction Data (ITD).
6	Trace External Message In.
7	Trace External Message Out.
8–16	Reserved for use of globally supported trace levels.
17–32	Reserved for object-specific trace levels.
33–64	Reserved for future use.

Trace File Interpretation

Interpretation of trace results is a manual process and can become quite complicated. Trace files can typically be interpreted at a process level simply by reading the ASCII version of the trace file. More detailed interpretation, however, may require that the ASCII trace file first be converted to binary or hexadecimal format. Trace file interpretation should only be attempted by qualified personnel.

Screen 1 Function Keys

The use of the following function keys on TRC screen 1 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Start a Trace — Displays TRC screen 2.
F3	Stop a Trace — Displays TRC screen 4.
F5	List Objects Being Traced — Displays TRC screen 5.

Screen 1

TRC screen 1 allows you to start a trace, stop a trace already in progress, or list the objects currently undergoing a trace. Depending upon which trace control function you select, the appropriate TRC screen is displayed. If you are starting a trace, TRC screen 2 is displayed. If you are stopping a trace, TRC screen 4 is displayed. If you are listing the objects currently undergoing a trace, TRC screen 3 is displayed.

TRC screen 1 is shown below. There are no individual fields on this screen; only the function keys are valid.

```
BASE24-KNL    TRACE CONTROL          LLLL          YY/MM/DD  HH:MM  01 OF 06
```

```
SELECT THE TRACE FUNCTION DESIRED:
```

```
F1 - START A TRACE
F3 - STOP A TRACE
F5 - LIST OBJECTS BEING TRACED
```

```
***** BASE24 *****
FILE DESTINATION:          NEW LOGICAL NETWORK ID:
F12-HELP
```

Screen 2 Function Keys

The use of the following function keys on TRC screen 2 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F2	Go to the Trace Secure ITD Screen — Displays TRC screen 6.
F15	Return to Page 1 — Displays TRC screen 1.

Screen 2

TRC screen 2 allows you to select the symbolic name of the process to which the TRACE command is sent. On this screen, you can also specify the location where the trace data is returned (that is, a terminal or trace file), the trace file size, and whether the trace can be wrapped back to the beginning of the trace file when the end of the current trace file is reached.

After entering the symbolic process name of the process to be traced and the trace file parameters, press the **F1** key to display TRC screen 3. This screen allows you to select the objects and object levels to be traced.

TRC screen 2 is shown below, followed by a description of each field on the screen.

```
BASE24-KNL    TRACE CONTROL                LLLL          YY/MM/DD  HH:MM  02 OF 06
```

```
ENTER THE SYMBOLIC NAME OF THE PROCESS TO SEND THE TRACE COMMAND TO.
```

```
PROCESS NAME:
```

```
TRACE FILE (SYS/VOL/SUBVOL):
```

```
TRACE FILE SIZE:    256
```

```
ALLOW WRAPPING:    N
```

```
***** BASE24 *****
```

```
FILE DESTINATION:    NEW LOGICAL NETWORK ID:
```

```
F1-ENTER  F2-GO TO TRACE SECURE ITD SCREEN  F15-RETURN TO PAGE 1 F12-HELP
```

PROCESS NAME — The symbolic name of the process to be traced. The name entered in this field must match a process symbolic name configured in the Process File (PRO).

Example: P1A^IAS01
Field Length: 1–16 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: Not applicable

TRACE FILE — The location of the trace file or the name of the terminal where response data for this trace is to be created.

If a terminal is used, the first four characters of the field must be TERM followed by a space and then the terminal name (for example, TERM \$LAN.#WIN01).

If a trace file is used, this field should contain only the location of where the trace file is to be created (that is, system, volume, and subvolume). It should not contain the name of the trace file itself, since trace files are automatically named, according to the following naming convention: TRC*ppdx*, where *ppd* is the PPD of the process being traced, and *x* is a number from 0 to 9. The Trace object attempts to locate the first file not already present starting with 0 all the way up to 9. If trace files from 0 through 9 already exist, the object purges the existing 0 file and creates a new one. The system continues writing to the same trace file until the current file fills up or you direct the trace to a different file.

If a trace file is created, the FUP COPY command can be used to analyze the contents of the trace file in different formats (for example, binary or hexadecimal).

Example: \B24.\$DATA.TRACE (system/volume/subvolume location)
TERM \$LAN.#WIN01 (terminal name)
Field Length: 1–25 alphanumeric characters
Required Field: Yes
Default Value: No default value
Data Name: Not applicable

TRACE FILE SIZE — The size, in pages, of the trace file to be created. Each page is 2048 bytes.

Field Length: 1–4 numeric characters
Required Field: Yes
Default Value: 256
Data Name: Not applicable

ALLOW WRAPPING — A flag indicating whether trace data can be wrapped back to the beginning of the trace file once the file is full or whether tracing should stop when the current trace file is full. If wrapping is allowed, existing trace data is overwritten each time the file wraps. Valid values are as follows:

Y = Yes, wrap to the beginning of the file when the trace file becomes full, overwriting earlier trace data.

N = No, do not wrap to the beginning of the file. When the trace file becomes full, stop the trace.

Field Length: 1 alphabetic character

Required Field: Yes

Default Value: N

Data Name: Not applicable

Screen 3 Function Keys

The use of the following function keys on TRC screen 3 varies from the standard function keys explained in the *BASE24 CRT Access Manual*. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects the objects within the ITS process to be traced. You must identify the objects to be traced by placing the value Y in the SELECT fields on the same line as each object prior to pressing this function key. You can select objects on multiple pages prior to pressing the F1 key.
F15	Return to Page 1 — Displays TRC screen 1.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of object entries if there are more entries than will fit on one screen.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of object entries if there are more entries than will fit on one screen.

Screen 3

TRC screen 3 allows you to select the individual objects within the specified process to be traced. The TRACE command is sent to each object selected. From this screen you can also indicate the trace levels supported for each of the selected objects. TRC screen 3 is displayed after entering a valid process name on TRC screen 2.

You select objects for tracing on this screen by placing the value Y in the SELECT field next to the object description of each object to be traced and pressing the **F1** key. You can use the **Shift-F7** and **Shift-F8** keys to view more object selections, if more than one screen of selections exist. Once you have selected all the objects you want to trace, press the **F1** key to execute the trace.

When a process receives the TRACE command, the Process Control object sends the Trace On member function request to the Trace object. The Trace object forwards the appropriate Start Trace member function requests to the objects identified in the request.

TRC screen 3 is shown below, followed by a description of each field on the screen.

```
BASE24-KNL  TRACE CONTROL  LLLL  YY/MM/DD  HH:MM  03 OF 06

TRACE ALL OBJECTS ON PAGE: N (Y/N)  ALL TRACE LEVELS: N (Y/N)

OBJECT DESCRIPTION  SELECT  TRACE LEVELS
```

```
***** BASE24 *****
FILE DESTINATION:  NEW LOGICAL NETWORK ID:
F1-ENTER  F15-RETURN TO PAGE 1  SF7-PREVIOUS PAGE  SF8-NEXT PAGE  F12-HELP
```

TRACE ALL OBJECTS ON PAGE — A flag indicating whether all objects currently displayed on the screen are to be traced. When you enter the value Y in this field and press the **F1** key, all objects currently displayed on the page are traced, regardless of whether you have selected individual objects in the SELECT field. When you enter the value N in this field and press the **F1** key, only those objects you select individually in the SELECT field will be traced.

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: N
Data Name: Not applicable

ALL TRACE LEVELS — A flag indicating whether all applicable levels are to be traced for each of the objects selected. When you enter the value Y in this field and press the **F1** key, all applicable trace levels in selected objects are traced regardless of the settings displayed in the TRACE LEVELS fields for each object on the page. When you enter the value N in this field and press the **F1** key, only the trace levels you specify in the TRACE LEVELS field for each object are traced.

Field Length: 1 alphabetic character
Required Field: Yes
Default Value: N
Data Name: Not applicable

OBJECT DESCRIPTION — The text description of an object that resides in the process on which the trace is to be performed. This description is retrieved from the OBJ. All objects related to the specified process in the Process/Object Relationship File (PORF) are displayed.

Field Length: System-protected
Occurs: Up to 12 times
Data Name: Not applicable

SELECT — A flag indicating whether the object displayed on the same line of the screen is to receive the TRACE command. Valid values are as follows:

Y = Yes, send the TRACE command to this object.
N = No, do not send the TRACE command to this object.

Field Length: 1 alphabetic character
Occurs: Up to 12 times
Required Field: Yes
Default Value: N
Data Name: Not applicable

TRACE LEVELS — A series of fields indicating the levels to be traced for each object. Trace levels are defined to objects using Object File (OBJ) screen 2. Up to 12 trace levels can be specified for each object. Trace levels can range from 1 to 64 and 99. A trace level of 99 indicates that all applicable levels are to be traced. For detailed descriptions of each trace level, see the Object File (OBJ) screen description provided elsewhere in this manual.

Field Length: 1–2 numeric characters
Occurs: Up to 12 times
Required Field: No
Default Value: No default value
Data Name: Not applicable

Screen 4 Function Keys

The use of the following function key on TRC screen 4 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The use of this function key is explained below.

The first column of information shows the BASE24 key. The second column describes the function that can be accomplished with this key.

Key	Description
F15	Return to Page 1 — Displays TRC screen 1.

Screen 4

TRC screen 4 allows you to stop a trace currently in progress. Traces are stopped on a process-by-process basis.

When you enter a symbolic process name in the PROCESS NAME field and press the **F1** key, the STOP TRACE command is sent to the Process Control object for the process. The Process Control object then sends the Trace Off member function request to the Trace object to stop all tracing for the process. The Trace object forwards the Trace Off member function request to every object currently involved in tracing.

TRC screen 4 is shown below, followed by a description of each field on the screen.

```
BASE24-KNL    TRACE CONTROL          LLLL          YY/MM/DD  HH:MM  04 OF 06
```

```
ENTER THE SYMBOLIC NAME OF THE PROCESS THAT SHOULD  
RECEIVE THE TRACE STOP COMMAND.
```

```
PROCESS NAME:
```

```
***** BASE24 *****  
FILE DESTINATION:      NEW LOGICAL NETWORK ID:  
F1-ENTER      F15-RETURN TO PAGE 1      F12-HELP
```

PROCESS NAME — The symbolic name of the process for which the trace is to be stopped. The name entered in this field must match the symbolic name of a process currently undergoing a trace.

Example:	P1A^IAS01
Field Length:	1–16 alphanumeric characters
Required Field:	Yes
Default Value:	No default value
Data Name:	Not applicable

Screen 5 Function Keys

The use of the following function key on TRC screen 5 varies from the standard function keys explained in the ***BASE24 CRT Access Manual***. The use of this function key is explained below.

The first column of information shows the BASE24 key. The second column describes the function that can be accomplished with this key.

Key	Description
F15	Return to Page 1 — Displays TRC screen 1.

Screen 5

TRC screen 5 allows you to send the TRACE LIST command to the process being traced. The TRACE LIST command lists the current status of a trace in progress. The response data to the TRACE LIST command includes the percentage of the trace completed thus far, whether file wrapping is allowed, and whether file wrapping has occurred. Trace list response information is logged in an Event Management System (EMS) event message.

When a process receives the TRACE LIST command, the Process Control object sends the Trace List member function request to the Trace object to initiate the trace list function.

TRC screen 5 is shown below, followed by a description of each field on the screen.

BASE24-KNL
TRACE CONTROL
LLLL
YY/MM/DD
HH:MM
05 OF 06

ENTER THE SYMBOLIC NAME OF THE PROCESS THAT SHOULD
RECEIVE THE TRACE LIST COMMAND.

PROCESS NAME:

***** BASE24 *****

FILE DESTINATION:
NEW LOGICAL NETWORK ID:

F1-ENTER
F15-RETURN TO PAGE 1
F12-HELP

PROCESS NAME — The symbolic name of the process to which the TRACE LIST command is to be sent. The name entered in this field must match the name of a process currently undergoing a trace.

Example:	P1A^IAS01
Field Length:	1–16 alphanumeric characters
Required Field:	Yes
Default Value:	No default value
Data Name:	Not applicable

Screen 6 Function Keys

The use of the following keys on TRC screen 6 varies from the standard function keys explained in the *BASE24 CRT Access Manual*. The use of these function keys is explained below.

The first column of information shows the BASE24 keys. The second column describes the functions that can be accomplished with these keys.

Key	Description
F1	Enter — Selects the objects within the ITS process to receive the TRACE ITD command. You must identify the objects to receive the TRACE ITD command by placing the value Y in the SELECT fields on the same line as each object prior to pressing this function key. You can select objects on multiple pages prior to pressing the F1 key.
F3	Go to Stop Trace Screen — Displays TRC screen 4.
F5	Go to List Trace Screen — Displays TRC screen 5.
F15	Return to Page 1 — Displays TRC screen 1.
Shift-F7	View Previous Page of Multi-page Entry — Displays the previous page of object entries if there are more entries than will fit on one screen.
Shift-F8	View Next Page of Multi-page Entry — Displays the next page of object entries if there are more entries than will fit on one screen.

Screen 6

TRC screen allows you to send the TRACE SECURE ITD command to selected objects within the specified process to be traced. The TRACE SECURE ITD command is used to trace security-related information (for example, PIN validation information) from the Internal Transaction Data (ITD).

This screen is displayed when you press the **F2** key on TRC screen 2. Any object defined to the process with an OBJECT TYPE field value of A on Object File (OBJ) screen 1 is displayed.

You select objects to receive the TRACE SECURE ITD command on this screen by placing the value Y in the SELECT field next to the name of each object. You can use the **Shift-F7** and **Shift-F8** keys to view more object selections, if more entries than will fit on one screen exist. Once you have selected all the objects which you want to receive the TRACE SECURE ITD command, press the **F1** key to execute the command.

When a process receives the TRACE SECURE ITD command, the Process Control object sends a Trace On Secure ITD member function request to the Trace object. The Trace object forwards the request to the specified objects (for example, Transaction Security object).

TRC screen 6 is shown below, followed by a description of each field on the screen.

BASE24-KNL
TRACE
LLLL
YY/MM/DD
HH:MM
06 OF 06

SELECT THE OBJECTS WHICH NEED TO RECEIVE THE TRACE SECURE ITD DATA COMMAND.

SELECT

OBJECT DESCRIPTION

***** BASE24 *****

FILE DESTINATION:NEW LOGICAL NETWORK ID:

F1-ENTERF15-RETURN TO PAGE 1SF7-PREVIOUS PAGESF8-NEXT PAGEF12-HELP

SELECT — A field indicating whether the object displayed on the same line of the screen is to receive the TRACE SECURE ITD command. Valid values are as follows:

- Y = Yes, send the TRACE SECURE ITD command to this object.
- N = No, do not send the TRACE SECURE ITD command to this object.

Field Length:	1 alphabetic character
Occurs:	Up to 12 times
Required Field:	Yes
Default Value:	N
Data Name:	Not applicable

OBJECT DESCRIPTION — The textual description of an object that resides in the specified process and has an OBJECT TYPE field value of A (security) on OBJ screen 1. All objects currently configured to the specified process with an object type of security will be displayed.

Field Length:	System-protected
Occurs:	Up to 12 times
Data Name:	Not applicable

Index

A

- AORF
 - see* Assign/Object Relationship File (AORF)
- Application Network Services object, definition of, [15-5](#)
- APRF
 - see* Assign/Process Type Relationship File (APRF)
- ASGN
 - see* Assign File (ASGN)
- Assign description field
 - on AORF screen 1, [3-3](#)
 - on APRF screen 1, [4-3](#)
- Assign File (ASGN)
 - default data, [2-3](#)
 - introduction, [2-1](#)
 - screen 1, [2-2](#)
- Assign Warmboot (AWB) screen
 - introduction, [5-1](#)
 - screen 1, [5-4](#)
 - screen 1 function keys, [5-3](#)
 - screen 2, [5-7](#)
 - screen 2 function keys, [5-6](#)
- Assign/Object Relationship File (AORF)
 - default data, [3-5](#)
 - function keys, [3-2](#)
 - introduction, [3-1](#)
 - screen 1, [3-3](#)
- Assign/Process Type Relationship File (APRF)
 - default data, [4-5](#)
 - function keys, [4-2](#)
 - introduction, [4-1](#)
 - screen 1, [4-3](#)
- AWB
 - see* Assign Warmboot (AWB) screen

B

- Basic files, [1-8](#)

C

- CCS
 - see* Configuration Control (CCS) screens
- Channel, definition of, [15-5](#)

- Configuration Control (CCS) screens
 - introduction, [6-1](#)
 - screen 1, [6-7](#)
 - screen 1 function keys, [6-6](#)
 - screen 2, [6-11](#)
 - screen 2 function keys, [6-10](#)

- Configuration files
 - Assign File, [2-1](#)
 - Assign/Object Relationship File, [3-1](#)
 - Assign/Process Type Relationship File, [4-1](#)
 - customer-configured data, [1-7](#)
 - EMT/Object Relationship File, [8-1](#)
 - EMT/Process Type Relationship File, [9-1](#)
 - Event Suppression File, [11-1](#)
 - Extended Memory Table File, [12-1](#)
 - Log Permission Process Partition File, [14-1](#)
 - Object File, [15-1](#)
 - preconfigured default data, [1-6](#)
 - Process File, [17-1](#)
 - Process Type File, [19-1](#)
 - Process/Object Relationship File, [18-1](#)
 - Server Class Type File, [21-1](#)
 - Server Class Type/Process Relationship File, [20-1](#)
 - Source Class Map File, [23-1](#)
- Customer-configured data, [1-7](#)

D

- Data entry hierarchy, [1-8](#)
- DBG
 - see* Debug (DBG) screen
- Debug (DBG) screen
 - introduction, [7-1](#)
 - screen 1, [7-3](#)

- Default data
 - AORF, [3-5](#)
 - APRF, [4-5](#)
 - EMT, [12-5](#)
 - EORF, [8-5](#)
 - EPRF, [9-5](#)
 - OBJ, [15-11](#)
 - PTF, [19-5](#)

E

- EMT
 - see* Extended Memory Table File (EMT)

EMT description field
 on EORF screen 1, [8-3](#)
 on EPRF screen 1, [9-3](#)

EMT Warmboot (EWB) screen
 introduction, [10-1](#)
 screen 1, [10-4](#)
 screen 1 function keys, [10-3](#)
 screen 2, [10-7](#)
 screen 2 function keys, [10-6](#)

EMT/Object Relationship File (EORF)
 default data, [8-5](#)
 function keys, [8-2](#)
 introduction, [8-1](#)
 screen 1, [8-3](#)

EMT/Process Type Relationship File (EPRF)
 default data, [9-5](#)
 function keys, [9-2](#)
 introduction, [9-1](#)
 screen 1, [9-3](#)

EORF
 see EMT/Object Relationship File (EORF)

EPRF
 see EMT/Process Type Relationship File (EPRF)

ESF
 see Event Suppression File (ESF)

Event Suppression File (ESF)
 introduction, [11-1](#)
 screen 1, [11-3](#)

EWB
 see EMT Warmboot (EWB) screen

Extended Memory Table File (EMT)
 default data, [12-5](#)
 introduction, [12-1](#)
 screen 1, [12-3](#)

F

File access, [1-4](#)

File dependencies, [1-9](#)

File relationships, [1-9](#)

G

Ghost object, definition of, [15-5](#)

H

Help screens, [1-12](#)

I

Integrated transaction services (ITS), definition of, [1-2](#)

ITS
 see Integrated transaction services (ITS), definition of

K

Kernel Product Menu, [1-4](#)

Kernel, definition of, [1-2](#)

L

LCONF Warmboot (LWB) screen
 function keys, [13-3](#)
 introduction, [13-1](#)
 screen 1, [13-4](#)

Log Permission Process Partition File (LPF)
 introduction, [14-1](#)
 screen 1, [14-3](#)

LPF
 see Log Permission Process Partition File (LPF)

LWB
 see LCONF Warmboot (LWB) screen

O

OBJ
 see Object File (OBJ)

Object description field
 on AORF screen 1, [3-4](#)
 on EORF screen 1, [8-4](#)
 on PORF screen 1, [18-4](#)
 on SCM screen 1, [23-4](#)

Object File (OBJ)
 default data, [15-11](#)
 introduction, [15-1](#)
 screen 1, [15-3](#)
 screen 2, [15-8](#)

P

Param Warmboot (PWB) screen
 function keys, [16-3](#)
 introduction, [16-1](#)
 screen 1, [16-4](#)

PORF
 see Process/Object Relationship File (PORF)

PRO
 see Process File (PRO)

Procedural, definition of, [1-2](#)

Process control function dependencies, [1-10](#)

Process control screens
 Assign Warmboot, [5-1](#)
 Configuration Control, [6-1](#)
 Debug, [7-1](#)
 EMT Warmboot, [10-1](#)
 LCONF Warmboot, [13-1](#)
 Param Warmboot, [16-1](#)
 Server Control, [22-1](#)
 Status, [24-1](#)
 Trace, [25-1](#)

- Process File (PRO)
 - introduction, [17-1](#)
 - screen 1, [17-2](#)
- Process type description field
 - on APRF screen 1, [4-4](#)
 - on EPRF screen 1, [9-4](#)
 - on PRO screen 1, [17-3](#)
- Process Type File (PTF)
 - default data, [19-5](#)
 - introduction, [19-1](#)
 - screen 1, [19-3](#)
- Process/Object Relationship File (PORF)
 - function keys, [18-2](#)
 - introduction, [18-1](#)
 - screen 1, [18-3](#)
- PTF
 - see* Process Type File (PTF)
- PWB
 - see* Param Warmboot (PWB) screen
- R**
- Relationship files, [1-9](#)
- S**
- SCM
 - see* Source Class Map File (SCM)
- SCS
 - see* Server Control (SCS) screen
- SCT
 - see* Server Class Type File (SCT)
- Server class type description field
 - on SPRF screen 1, [20-3](#)
- Server Class Type File (SCT)
 - introduction, [21-1](#)
 - screen 1, [21-3](#)
- Server Class Type/Process Relationship File (SPRF)
 - function keys, [20-2](#)
 - introduction, [20-1](#)
 - screen 1, [20-3](#)
- Server Control (SCS) screen
 - introduction, [22-1](#)
 - screen 1, [22-3](#)
 - screen 2, [22-6](#)
 - screen 2 function keys, [22-5](#)
- Simulator object, definition of, [15-5](#)
- Source Class Map File (SCM)
 - introduction, [23-1](#)
 - screen 1, [23-3](#)
- SPRF
 - see* Server Class Type/Process Relationship File (SPRF)
- SSID subsystem description field
 - on ESF screen 1, [11-4](#)
 - on LPF screen 1, [14-4](#)
- STAT
 - see* Status (STAT) screen
- Status (STAT) screen
 - introduction, [24-1](#)
 - screen 1, [24-3](#)
 - screen 2, [24-5](#)
 - screen 2 function keys, [24-4](#)
 - status command response data, [24-7](#)
- T**
- Trace Control (TRC) screens
 - introduction, [25-1](#)
 - screen 1, [25-6](#)
 - screen 1 function keys, [25-5](#)
 - screen 2, [25-8](#)
 - screen 2 function keys, [25-7](#)
 - screen 3, [25-12](#)
 - screen 3 function keys, [25-11](#)
 - screen 4, [25-16](#)
 - screen 4 function keys, [25-15](#)
 - screen 5, [25-19](#)
 - screen 5 function keys, [25-18](#)
 - screen 6, [25-22](#)
 - screen 6 function keys, [25-21](#)
 - trace file interpretation, [25-4](#)
 - trace levels, [25-3](#)
- TRC
 - see* Trace Control (TRC) screens
- Y**
- Yang object, definition of, [15-5](#)

ACI Worldwide Inc.

Index by Field Name

Note: Unlabeled fields are not included in this index; they appear by subject in the main index.

A

ALL TRACE LEVELS, [25-13](#)
ALLOW WRAPPING, [25-10](#)
ASSIGN DESCRIPTION, [2-2](#)
ASSIGN NAME
 AORF, [3-3](#)
 APRF, [4-3](#)
 ASGN, [2-2](#)
 AWB screen 1, [5-5](#)

C

CONTROL FILE ASSIGN NAME, [21-4](#)
CONTROL FILE NUMBER, [21-4](#)
CPU, [6-8](#)

E

EMT DESCRIPTION, [12-4](#)
EMT NAME
 CCS screen 2, [6-12](#)
 EMT, [12-3](#)
 EORF, [8-3](#)
 EPRF, [9-3](#)
 EWB screen 1, [10-5](#)
EMT WARMBOOT ALLOWED, [12-4](#)
EVENT NUMBER, [11-4](#)

F

FIRST SSID, [14-3](#)
FUNCTION DATA, [22-4](#)
FUNCTION NAME, [22-3](#)

G

GENERAL COMMAND FLAG, [15-6](#)

L

LOG STATE, [11-4](#)
LPP ASSIGN NAME, [14-4](#)

N

NUMBER OF EVENTS TO SKIP, [11-5](#)

O

OBJECT DESCRIPTION
 AWB screen 2, [5-8](#)
 EWB screen 2, [10-9](#)
 LWB, [13-5](#)
 OBJ screen 1, [15-5](#)
 PWB, [16-5](#)
 SCS screen 2, [22-7](#)
 STAT screen 2, [24-6](#)
 TRC screen 3, [25-13](#)
 TRC screen 6, [25-24](#)
OBJECT ID, [15-3](#)
OBJECT NAME
 AORF, [3-4](#)
 EORF, [8-4](#)
 OBJ screen 1, [15-3](#)
 PORF, [18-4](#)
 SCM, [23-3](#)
OBJECT TYPE, [15-7](#)

P

PPD NAME, [17-2](#)
PRIORITY, [6-8](#)
PROCESS NAME
 CCS screen 1, [6-8](#)
 DBG, [7-3](#)
 PORF, [18-3](#)
 PRO, [17-2](#)
 SCS screen 1, [22-3](#)
 SPRF, [20-4](#)
 STAT screen 1, [24-3](#)
 TRC screen 2, [25-8](#)
 TRC screen 4, [25-17](#)
 TRC screen 5, [25-19](#)
PROCESS TYPE
 APRF, [4-4](#)
 EPRF, [9-4](#)
 PRO, [17-3](#)
 PTF, [19-3](#)
PROCESS TYPE DESCRIPTION, [19-4](#)
PROGRAM LOCATION, [6-8](#)

Q

QUEUE FLAG, [17-3](#)

R

REPORT LOCATION, [6-9](#)

S

SELECT

AWB screen 1, [5-4](#)

AWB screen 2, [5-8](#)

CCS screen 2, [6-12](#)

EWB screen 1, [10-5](#)

EWB screen 2, [10-8](#)

LWB, [13-5](#)

PWB, [16-5](#)

SCS screen 2, [22-7](#)

STAT screen 2, [24-6](#)

TRC screen 3, [25-13](#)

TRC screen 6, [25-23](#)

SERVER CLASS TYPE

SCT, [21-3](#)

SPRF, [20-3](#)

SERVER CLASS TYPE DESCRIPTION, [21-4](#)

SOURCE CLASS, [23-3](#)

SSID, [11-3](#)

STATUS COMMAND FLAG, [15-6](#)

SUPPORTED BY EMT BUILD, [12-4](#)

T

TIME INTERVAL, [11-5](#)

TIME UNIT, [11-5](#)

TRACE ALL OBJECTS ON PAGE, [25-13](#)

TRACE FILE, [25-9](#)

TRACE FILE SIZE, [25-9](#)

TRACE LEVEL, [15-8](#)

TRACE LEVELS, [25-14](#)

W

WARMBOOT LCONF FLAG, [15-5](#)

WARMBOOT PARAM FLAG, [15-6](#)

Index by Data Name

A

ALTKEY.LOG-ST, [11-5](#)
ALTKEY.OBJ-ID, [15-5](#)
ALTKEY.OBJ-NAM, [23-4](#)
ALTKEY.PRO-TYP, [17-3](#)
ASGN-DESCR
 AORF, [3-3](#), [4-3](#)
 ASGN, [2-2](#)

C

CNTL-FILE-ASGN-NAM, [21-4](#)
CNTL-FILE-NUM, [21-4](#)

D

DELTA-TIME-MS, [11-5](#)

E

EMTB-SUPPORT, [12-4](#)
EMT-DESCR, [8-3](#), [9-3](#), [12-4](#)
EMT-WB-ALWD, [12-4](#)

G

GENRL-CMD-FLG, [15-7](#)

L

LCONF-WB-FLAG, [15-6](#)
LPP-ASGN-NAM, [14-4](#)

O

OBJ-DESCR, [3-4](#), [8-4](#), [15-5](#), [18-4](#), [23-4](#)
OBJ-TYP, [15-7](#)

P

PARAM-WB-FLAG, [15-6](#)
PPD-NAM, [17-3](#)
PRIKEY.ALTKEY.OBJ-NAM
 AORF, [3-4](#)
 EORF, [8-4](#)
 PORF, [18-4](#)

PRIKEY.ALTKEY.PRO-NAM, [20-4](#)
PRIKEY.ALTKEY.PRO-TYP
 APRF, [4-4](#)
 EORF, [9-4](#)
PRIKEY.ASGN-NAM
 AORF, [3-3](#)
 APRF, [4-3](#)
 ASGN, [2-2](#)
PRIKEY.EMT-NAM
 EMT, [12-3](#)
 EORF, [8-3](#)
 EPRF, [9-3](#)
PRIKEY.EVT-NUM, [11-4](#)
PRIKEY.OBJ-NAM, [15-3](#)
PRIKEY.PRO-NAM
 PORF, [18-3](#)
 PRO, [17-2](#)
PRIKEY.SRC-CLAS, [23-3](#)
PRIKEY.SRV-CLAS-TYP
 SCT, [21-3](#)
 SPRF, [20-3](#)
PRIKEY.SSID, [11-3](#)
PRIKEY.SSID-FIRST, [14-3](#)
PRO-TYP, [19-3](#)
PRO-TYP-DESCR, [4-4](#), [9-4](#), [17-3](#), [19-4](#)

Q

QUEUE-FLG, [17-3](#)

S

SKIP-CNT-MAX, [11-5](#)
SRV-CLAS-TYP-DESCR, [20-3](#), [21-4](#)
SSID.Z-NUMBER, [11-4](#), [14-4](#)
STAT-CMD-FLAG, [15-6](#)

T

TRC-LVL, [15-10](#)

ACI Worldwide Inc.